

Mycellius — Séance 6 (4h) :

API pour front

API propre pour front : DTO/Mapper + validation + erreurs standard + liste avec pagination + tags + recherche simple titre + Swagger/OpenAPI

Objectif de la séance

À la fin de cette séance, tu dois avoir une API réellement “front-ready” :

1. Un contrat HTTP propre et stable :

- en entrée : un DTO de création validé (Jakarta Validation)
- en sortie : un DTO de réponse (tu n'exposes plus directement ni le domaine, ni les entités JPA)

2. Un mapping industrialisé avec MapStruct :

- DTO → Domaine et Domaine → DTO (couche web)
- Domaine → Entités JPA et Entités JPA → Domaine (couche persistance)

3. Une gestion “propre” des tags en base :

- tables tag + association page_tag
- endpoints qui renvoient tags/pages avec tags

4. Des endpoints exploitables par un front :

- liste paginée
- recherche simple par titre

5. Une documentation API immédiatement consultable (Swagger/OpenAPI) et une gestion d'erreurs cohérente, y compris pour les erreurs de validation.

Découpage indicatif de la séance (4h)

1. Rappel séance 5 + objectif séance 6 (10–15 min)
2. Pourquoi DTO + validation : stabiliser le contrat HTTP (20–30 min)
3. Ajouter Jakarta Validation + définir les DTO (45–55 min)
4. Ajouter MapStruct + mapping DTO ↔ Domaine (45–55 min)
5. Modèle DB des tags + migrations Flyway (35–45 min)
6. Entités JPA tags + relation avec WikiPageEntity (30–40 min)
7. Mapping Domaine ↔ Entités + “résolution” des tags (find or create) (40–50 min)
8. Endpoints : pagination + recherche titre + liste tags (30–40 min)
9. Erreurs standard + erreurs de validation DTO (20–30 min)
10. Tests Postman + Swagger + rituel Git/CI dev → test (20–30 min)

Pré-requis (à vérifier avant de commencer)

- Tu es à la fin de la séance 5 : MySQL Docker + Flyway + JPA OK, endpoints `/api/v1/pages` persistants (POST + GET), et tu sais redémarrer l’API et vérifier la persistance.
- Ton handler d’erreurs type `ApiErrorResponse/ApiExceptionHandler` existe déjà (séance 4).
- La CI GitHub continue de faire `mvn -f api/pom.xml test` sans DB (on garde cette contrainte).

Précondition de test (important, comme en séance 5)

Pourquoi : tu veux prouver que tu ne casses pas la persistance en refactorant.

1. Démarre MySQL + l’API.
2. Crée une page via Postman sur POST `/api/v1/pages`.
3. Fais immédiatement un GET `/api/v1/pages/{id}` et note l’id (ex : PAGE-DB-001).
4. Tu gardes cette page comme “témoignage” avant/après refactor.

Préparer ton environnement (où tu te places, ce que tu dois voir)

- Ouvre un terminal dans le dossier du projet (racine du repo).
- Tu dois avoir le docker-compose de la DB (celui de la séance 5, généralement dans infra/db/docker-compose.yml).
- Tu dois avoir l'API prête à être lancée (Spring Boot) et configurée pour pointer sur MySQL Docker (propriétés de séance 5).

1 - Démarrer MySQL (Docker)

But : avoir la base MySQL "up" avant de lancer l'API.

- Dans le terminal, lance la DB (adapte le chemin si nécessaire) :
 - Si ton compose est dans infra/db/ :
 - > Va dans ce dossier puis démarre : `docker compose up -d`
 - > ou depuis la racine : `docker compose -f infra/db/docker-compose.yml up -d`
- Vérifie que le container tourne : `docker ps`
 - > Tu dois voir un container MySQL en statut "Up".
- Optionnel mais recommandé : vérifier que MySQL répond :
 - > `docker logs <nom_du_container>` (tu dois voir "ready for connections")
 - > Ou un test de connexion si tu l'as déjà utilisé en séance 5.

1 - Démarrer l'API (Spring Boot)

But : l'API doit se connecter à MySQL et jouer les migrations Flyway.

- Dans un second terminal (ou dans IntelliJ), démarre l'API :
 - > terminal (exemple) : `mvn -f api/pom.xml spring-boot:run`
 - > ou IntelliJ : Run "ApiApplication" (ou équivalent)
- Sur la console, vérifie 2 choses :
 - > Spring Boot démarre sans erreur de connexion DB
 - > applique (ou vérifie) les migrations
 - => Résultat attendu : l'API répond sur <http://localhost:8080/api/health>

2- Créer une page via Postman (POST /api/v1/pages)

But : créer une ligne en base (wiki_page) et obtenir un id exploitable et on vérifie que le POST fonctionne sur la bonne URL

2.1 Configurer la requête Postman

- Ouvre Postman
- Crée une nouvelle requête
- Mets la méthode : POST

URL : <http://localhost:8080/api/v1/pages>

2.2 Ajouter les headers

- Onglet "Headers" :
 - > Key: Content-Type
 - > Value: application/json

2.3 Écrire le JSON (Body)

- Onglet "Body" → "raw" → JSON

> Colle un JSON simple (adapte si ton endpoint n'accepte pas encore tags, ou si ton DTO actuel est différent). Pour une précondition "séance 5", fais au plus simple :

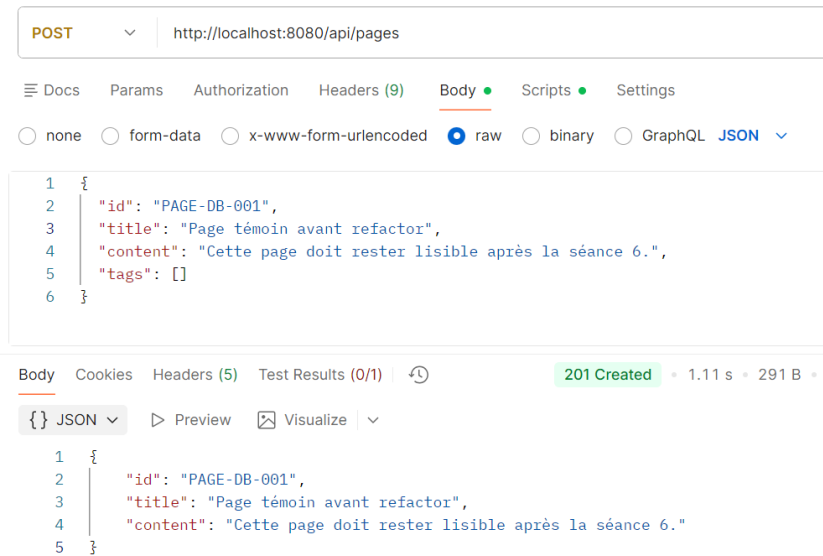
```
{
  "id": "PAGE-DB-001",
  "title": "Page témoin avant refactor",
  "content": "Cette page doit rester lisible après la séance 6.",
  "tags": []
}
```

2.4 Envoyer

- Clique "Send".

Résultat attendu :

```
{
  "id": "PAGE-DB-001",
  "title": "Page témoin avant refactor",
  "content": "Cette page doit rester lisible après la séance 6.",
  "tags": []
}
```



3 - Vérifier immédiatement la lecture (preuve de persistance)

Dans Postman :

Crée une nouvelle requête

Méthode : GET

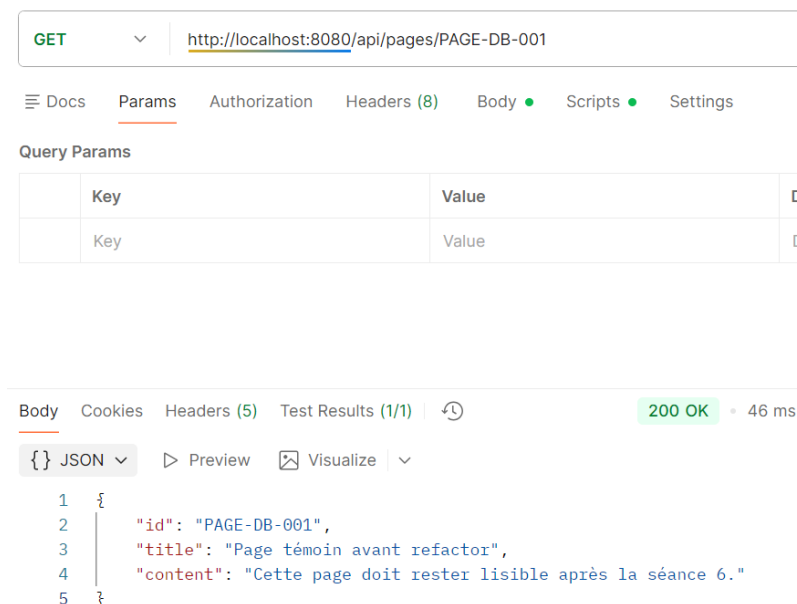
URL : <http://localhost:8080/api/pages/PAGE-DB-001>

Clique “Send”.

Résultat attendu :

Status : 200 OK

Body : tu retrouves "title": "Page témoin avant refactor" et le contenu.



4 - “Témoignage” (ce que tu dois garder avant le refactor)

Dans ton TP (ou dans un bloc-notes), écris exactement :

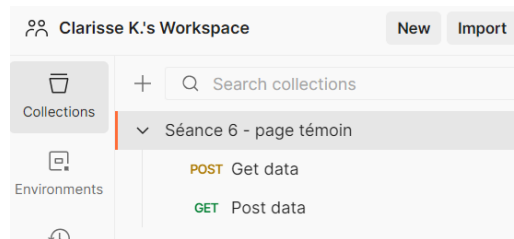
ID témoin avant refactor : PAGE-DB-001

URL GET témoin : /api/pages/PAGE-DB-001

Dans Postman :

Mets tes 2 requêtes (POST + GET) dans une collection nommée :

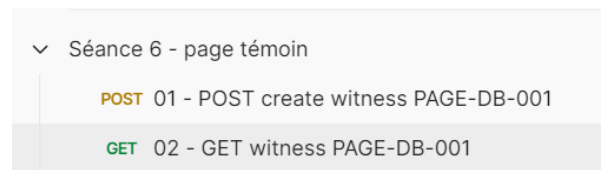
“Séance 6 — Précondition (page témoin)”



Renomme les requêtes :

01 - POST create witness PAGE-DB-001

02 - GET witness PAGE-DB-001



(Optionnel mais très utile) : copie-colle la réponse du GET dans ton TP, ou fais une capture.

```
{
  "id": "PAGE-DB-001",
  "title": "Page témoin avant refactor",
  "content": "Cette page doit rester lisible après la séance 6."
}
```

=> La précondition est validée : tu as une page témoin persistée et relisible. On peut démarrer la séance 6 !!!! oh yeah !!! C'est parti !!!

Analogie de la séance 6 : Dossier administratif — guichet, règles, archives

Que fait-on en séance 6 exactement et pourquoi ?

En séance 5, tu as créé l'archive (base de données) et la façon d'y ranger des dossiers (JPA/Flyway).

En séance 6, on professionnalise le "guichet" : le citoyen doit remplir le bon formulaire, on vérifie qu'il est correct, puis on l'enregistre proprement dans les archives.

Correspondances :

- Le DTO = le formulaire officiel

> Le citoyen ne peut pas arriver avec "un texte libre". Il remplit un formulaire qui a un format précis (champs attendus).

- Validation Jakarta = la vérification du dossier au guichet :

- > "Il manque la pièce d'identité" → champ obligatoire manquant
- > "Le numéro n'a pas le bon format" → pattern invalide
- > Si ça ne va pas : on refuse le dossier avec une liste claire des erreurs.

- Contrôleur = agent d'accueil

> Il reçoit le formulaire, contrôle, transmet au service concerné.

- Service métier = service instructeur

> Il applique les règles métier : acceptation, refus, recherche, logique "page inexistante", etc.

- Persistance = archives

C'est l'endroit où l'on stocke réellement. Les entités JPA correspondent au format "archivage".

- MapStruct = la procédure interne de transcription

Le formulaire (DTO) n'est pas stocké tel quel : il faut le transcrire en format interne (domaine) puis en format archivage (entity).

MapStruct automatise cette transcription proprement, sans recoder partout.

Phrase clé :

DTO = formulaire,

Validation = contrôle du dossier,

Service = instruction,

Entity = format d'archivage,

MapStruct = transcription.

1. Préparer la séance 6 : dépendances Maven (Jakarta Validation + MapStruct + OpenAPI)

On va préparer le projet (ajouter Validation Jakarta + MapStruct)

Pourquoi on commence par ça :

tant que les dépendances ne sont pas en place, tu ne peux pas :

- (1) déclencher @Valid sur tes DTO,
- ni (2) générer les mappers MapStruct à la compilation.

1.1 Pour cela, on va ajouter la dépendances pour la validation Jakarta (DTO)

Fichier : api/pom.xml

But : activer @Valid et toutes les annotations jakarta.validation.* côté web.

Ajoute dans <dependencies> :

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Ce que ça change :

Sans cette dépendance, tu peux écrire @NotBlank, mais Spring ne déclenche pas le 400 "validation" comme tu l'attends.

Ca veut dire quoi ?

Concrètement, si on reprend l'analogie "dossier administratif" :

La dépendance spring-boot-starter-validation, c'est l'équivalent de "l'agent au guichet + sa checklist officielle" qui sait contrôler un dossier.

Avec la dépendance (guichet équipé)

Tu arrives avec un formulaire (DTO) où certaines cases sont obligatoires :

- > @NotBlank = "champ obligatoire"
- > @Pattern = "format imposé"
- > @Size = "taille/longueur autorisée"

Le guichet (Spring) sait lire ces consignes et fait le contrôle immédiatement à l'entrée, avant d'envoyer le dossier au service instructeur. Si c'est invalide, il te renvoie :

- > 400 Bad Request
- > avec le détail : "title est obligatoire", etc.

Sans la dépendance (guichet pas équipé)

Tu peux toujours écrire @NotBlank sur le formulaire, mais c'est comme si tu avais imprimé "champ obligatoire" sur le papier sans qu'il y ait un agent formé ni une procédure pour vérifier.

Résultat concret côté API :

- > Le contrôleur reçoit quand même le JSON, même s'il est invalide (par exemple title="").
- > Le contrôle automatique ne se fait pas.
- > Donc Spring ne déclenche pas l'erreur "validation" (pas de 400 automatique).
- > Ton code continue, et l'erreur (si elle arrive) se produit plus loin, souvent de façon moins propre :
 - > soit tu enregistres des données incohérentes,
 - > soit tu obtiens une exception plus tard (ex. NullPointerException, contrainte DB),
 - > soit tu dois écrire toi-même des if (title == null || title.isBlank()) throw ... partout.

En une phrase très simple

La dépendance Validation = "le guichet qui sait réellement contrôler les formulaires". Sans elle, les annotations @NotBlank, @Size sont juste des "mentions sur le papier" que personne n'applique automatiquement.

1.2 Et on va ajouter MapStruct (mapper automatique)

Toujours dans api/pom.xml :

```
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>1.5.5.Final</version>
</dependency>
```

Puis, toujours dans api/pom.xml, activer le “processor” au build (Maven compiler plugin) dans <build><plugins> :

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <annotationProcessorPaths>
      <path>
        <groupId>org.mapstruct</groupId>
        <artifactId>mapstruct-processor</artifactId>
        <version>1.5.5.Final</version>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
```

Pourquoi ce bloc est obligatoire :

MapStruct génère du code à la compilation. Sans “processor”, tu n’auras pas les implémentations et ton injection Spring échouera.

Mais ça veut dire quoi exactement encore ?

On reprend l'analogie du “dossier administratif”

MapStruct c’est la “procédure de transcription” officielle qui transforme ton formulaire (DTO) en format interne (domaine) puis en format d’archivage (entity).

MapStruct (la librairie) = le manuel de transcription

Ajouter la dépendance mapstruct, c’est comme mettre à disposition un manuel qui décrit comment on doit recopier les informations entre deux formulaires différents.

Exemple :

Formulaire citoyen (DTO) : title, content, tags

Dossier interne (domaine) : WikiPage, Tag

Format archives (entity) : WikiPageEntity, TagEntity

Le “processor” MapStruct = le service qui fabrique les formulaires pré-remplis

Le mapstruct-processor, c’est le “service interne” qui, à chaque nouvelle version des formulaires, fabrique automatiquement les versions pré-remplies (les classes de mapping concrètes).

En informatique, ça veut dire :

Tu écris une interface WikiPageDtoMapper (le plan)

À la compilation, MapStruct génère une classe réelle du type WikiPageDtoMapperImpl

(l’employé qui fait le travail)

Sans processor : tu as le plan, mais aucun employé n'existe

Concrètement, sans mapstruct-processor (ou sans annotation processing activé), tu n'auras pas la classe générée.

Dans l'analogie :

- => Tu as rédigé une procédure "recopier champ A vers champ B"
- => Mais personne n'a été recruté/affecté pour appliquer la procédure
- => Donc le guichet ne peut pas "transcrire" automatiquement

"Injection Spring échouera" = Spring cherche un agent, mais il n'existe pas

Quand tu écris : `@Mapper(componentModel = "spring")`

- tu demandes : "ce mapper est un agent du guichet, Spring doit pouvoir l'injecter".
- Spring va donc essayer de trouver un bean du type `WikiPageDtoMapper`.

Avec processor :

- MapStruct génère `WikiPageDtoMapperImpl`
- Spring détecte cette classe comme bean
- L'injection fonctionne (le guichet a bien son agent de transcription)

Sans processor :

- `WikiPageDtoMapperImpl` n'existe pas
- Spring ne trouve aucun bean
- Au démarrage, tu obtiens typiquement une erreur du style :
- "No qualifying bean of type 'WikiPageDtoMapper' available"

En une phrase très simple

- MapStruct = la procédure de transcription.
- Le processor = l'employé qui est généré automatiquement pour appliquer cette procédure.
- Sans processor, l'employé n'existe pas, donc Spring ne peut pas l'injecter, et l'application plante au démarrage.

Mais c'est quoi un **bean** ?

Un "bean" Spring, c'est un agent officiel (ou un service officiel) qui fait partie de l'administration et que le système sait "appeler" automatiquement.

Ce que Spring fait, en version administration

Spring, c'est le bâtiment administratif avec son annuaire interne.

Dans cet annuaire, il liste tous les agents disponibles et leurs rôles :

- agent d'accueil (contrôleur)
- service instructeur (service métier)
- service d'archives (repository/persistance)
- service de transcription (mapper)

Chaque agent/service enregistré dans l'annuaire = un "bean".

À quoi sert un bean ?

Quand un agent a besoin d'un autre service, il ne va pas "le recruter à la main".

Il demande à l'administration : "donne-moi le service X".

Exemple concret :

Ton contrôleur (agent d'accueil) a besoin du service WikiService (service instructeur).

Spring lui "fournit" automatiquement ce service parce qu'il est enregistré comme bean.

Pourquoi ça s'appelle "bean" ?

C'est juste le terme Spring pour dire :

"un objet que Spring crée, configure, et gère pour toi".

Donc :

- tu ne fais pas `new WikiService()` dans ton contrôleur
- Spring crée WikiService au démarrage et le donne au contrôleur

Comment un objet devient bean ?

Dans l'analogie : comment quelqu'un devient "agent officiel" ?

En pratique, dans Spring, il suffit souvent d'une annotation :

- > `@RestController` : "cet objet est un agent d'accueil"
- > `@Service` : "cet objet est un service instructeur"
- > `@Repository` : "cet objet est un service d'archives"
- > `@Mapper(componentModel="spring")` : "cet objet est un service officiel de transcription"

Spring scanne le projet, trouve ces annotations, et inscrit ces objets dans son annuaire.

Lien direct avec ton problème MapStruct

Quand tu dis : "injection Spring échoue", ça signifie :

“Le guichet essaie de trouver l’agent X dans l’annuaire, mais il n’existe pas.”

Dans MapStruct :

- => tu declares l’agent (“il existe un service de transcription”) via l’interface WikiPageDtoMapper
- => mais sans processor, l’implémentation concrète n’est pas générée
- => donc Spring n’a personne à inscrire dans l’annuaire
- => donc l’injection échoue

Pour résumer

Un bean = un agent/service officiel enregistré dans l’annuaire de l’administration (Spring), que Spring peut fournir automatiquement aux autres.

Test à faire (obligatoire avant d’avancer)

Depuis la racine du projet : mvn -f api/pom.xml test

Résultat attendu : BUILD SUCCESS (aucune erreur de compilation).

1.3 Ajouter Swagger/OpenAPI (preuve “front-ready”)

On va modifier api/pom.xml, dans la section <dependencies>, ajoute :

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.15</version>
</dependency>
```

puis checkpoint build depuis la racine du dépôt :

```
mvn -f api/pom.xml test
```

Ce checkpoint reprend le rituel des séances : on compile/test avant de commit.

Résultat attendu : BUILD SUCCESS.

1.4 Vérification rapide Swagger (recommandée pour valider la valeur “front-ready”)

Démarre l’API puis ouvre dans le navigateur :

<http://localhost:8080/swagger-ui/index.html> et

<http://localhost:8080/v3/api-docs>

Résultat attendu : une page Swagger UI s’affiche (même si elle est encore “pauvre” en endpoints pour l’instant, c’est normal).

The image displays two browser windows. The left window shows the Swagger UI interface at <http://localhost:8080/v3/api-docs>. It features an 'OpenAPI definition' section with a 'Servers' dropdown set to 'http://localhost:8080 - Generated server url'. Below this, there are expandable sections for 'wiki-page-controller' and 'health-controller'. The 'GET /api/health' endpoint is selected. The 'Schemas' section is also expanded, showing 'CreatePageRequest', 'Tag', and 'WikiPage'.

The right window shows the raw OpenAPI JSON definition at <http://localhost:8080/v3/api-docs>. The JSON is as follows:

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "OpenAPI definition",
    "version": "v0"
  },
  "servers": [
    {
      "url": "http://localhost:8080",
      "description": "Generated server url"
    }
  ],
  "paths": {
    "/api/pages": {
      "post": {
        "tags": [
          "wiki-page-controller"
        ],
        "operationId": "createPage",
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/CreatePageRequest"
              }
            }
          },
          "required": true
        },
        "responses": {
          "200": {
            "description": "OK",
            "content": {
              "*/*": {
                "schema": {
                  "$ref": "#/components/schemas/WikiPage"
                }
              }
            }
          }
        }
      }
    },
    "/api/pages/{id}": {
      "get": {
        "tags": [
          "wiki-page-controller"
        ],
        "operationId": "getPageById",
        "parameters": [
          {
            "name": "id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "string"
            }
          }
        ]
      }
    }
  }
}
```

Below the JSON, there is a table for 'Query Params' with columns 'Key', 'Value', and 'Description'. The table is currently empty.

At the bottom, the 'Body' section is expanded, showing the JSON response. The response is a 200 OK status with a 305 ms response time and a 1.8 KB body size. The JSON body is:

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "OpenAPI definition",
    "version": "v0"
  },
  "servers": [
    {
      "url": "http://localhost:8080",
      "description": "Generated server url"
    }
  ],
  "paths": {
    "/api/pages": {
      "post": {
        "tags": [
          "wiki-page-controller"
        ],
        "operationId": "createPage",
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/CreatePageRequest"
              }
            }
          },
          "required": true
        },
        "responses": {
          "200": {
            "description": "OK",
            "content": {
              "*/*": {
                "schema": {
                  "$ref": "#/components/schemas/WikiPage"
                }
              }
            }
          }
        }
      }
    },
    "/api/pages/{id}": {
      "get": {
        "tags": [
          "wiki-page-controller"
        ],
        "operationId": "getPageById",
        "parameters": [
          {
            "name": "id",
            "in": "path",
            "required": true,
            "schema": {
              "type": "string"
            }
          }
        ]
      }
    }
  }
}
```

2. Arborescence cible : où mettre quoi (et pourquoi)

But : rendre lisible “HTTP vs Métier vs DB”. C’est l’idée de séparation expliquée en séance 4.

On a déjà `fr.mycellius.domain.*` et `fr.mycellius.persistence.*` (séance 5).

Vérification à faire

Dans IntelliJ (ou dans l’explorateur), tu dois voir ces nouveaux packages apparaître sous `fr.mycellius`.

On ajoute/structure :

Côté web (HTTP)

`fr.mycellius.web.dto` :

contrats HTTP (request/response)

`fr.mycellius.web.mapper` :

MapStruct DTO ↔ Domaine

`fr.mycellius.web.controller` :

contrôleurs REST

`fr.mycellius.web` (ou `fr.mycellius.web.advice`) :

exceptions → HTTP (`ApiExceptionHandler`)

Côté persistance (DB)

`fr.mycellius.persistence.entity` :

entités JPA (DB)

`fr.mycellius.persistence.jpa` :

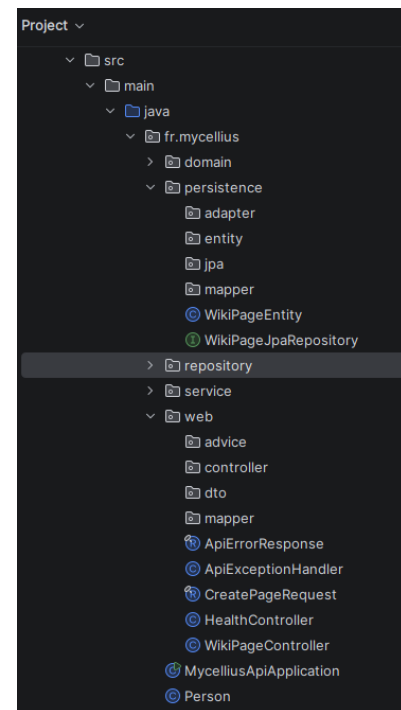
repositories Spring Data

`fr.mycellius.persistence.mapper` :

MapStruct Domaine ↔ Entity

`fr.mycellius.persistence.adapter` :

implémentation DB (`DatabaseWikiRepository`) (séance 5)



Côté “contrat de stockage”

`fr.mycellius.repository` :

interfaces “contrat de stockage” (`WikiRepository`) (existe déjà)

3. Créer les DTO + ajouter la validation Jakarta

3.1 DTO Tag en entrée

Créer le fichier : api/src/main/java/fr/mycellius/web/dto/TagRequest.java

```
package fr.mycellius.web.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record TagRequest(
    @NotBlank(message = "Le tag est obligatoire")
    @Size(min = 1, max = 30, message = "Le tag doit faire entre 1 et 30 caractères")
    String name
) {}
```

Ce qu'il faut comprendre :

@NotBlank protège l'API contre null, "", " "

@Size t'évite d'accepter des tags absurdes (ex : 2000 caractères)

Test minimal (sans Postman, juste compilation), lance :

```
mvn -f api/pom.xml test
```

Résultat attendu : BUILD SUCCESS.

Par analogie :

TagRequest + ses annotations, c'est la "case Tag" du formulaire administratif, avec les règles de contrôle appliquées au guichet avant d'envoyer le dossier au service instructeur ou aux archives.

Pour rappel:

=> Guichet (Controller) vérifie et transmet.

=> Service instructeur (Service) applique les règles métier.

=> Archives (Persistence/Repository) stockent et relisent.

3.2 DTO de création de page (entrée)

Fichier : api/src/main/java/fr/mycellius/web/dto/CreateWikiPageRequest.java

```
package fr.mycellius.web.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;
```



```

import jakarta.validation.constraints.Size;

import java.util.List;

public record CreateWikiPageRequest(

    @NotBlank(message = "L'id est obligatoire")
    @Pattern(regexp = "PAGE-\\d{3}", message = "Format d'id attendu : PAGE-001")
    String id,

    @NotBlank(message = "Le titre est obligatoire")
    @Size(min = 3, max = 120, message = "Le titre doit faire entre 3 et 120 caractères")
    String title,

    @NotBlank(message = "Le contenu est obligatoire")
    @Size(max = 10000, message = "Le contenu est trop long (max 10000)")
    String content,

    @Valid
    List<TagRequest> tags
) {}

```

Par analogie :

- > CreateWikiPageRequest = le formulaire principal “Dossier de création de page Wiki”.
- > id : numéro de dossier. @Pattern impose un format standard (PAGE-001), comme un numéro de dossier administratif.
- > title, content : champs obligatoires, avec limites, comme des cases du formulaire.
- > tags : une sous-partie du dossier (annexe).
- > Le point clé :
@Valid sur tags = “le guichet vérifie aussi les annexes” ou “valide aussi chaque TagRequest”
Sans @Valid, le guichet ne contrôle pas le contenu des annexes (les TagRequest), et laisse passer des tags invalides.

Relance un test : `mvn -f api/pom.xml test`

3.3 DTO de réponse (sortie)

Fichier : `api/src/main/java/fr/mycellius/web/dto/WikiPageResponse.java`

```

package fr.mycellius.web.dto;

import java.time.Instant;
import java.util.List;

```

```

public record WikiPageResponse(
    String id,
    String title,
    String content,
    List<String> tags,
    Instant createdAt
) {}

```

Pourquoi List<String> en sortie :

- côté front c'est plus simple à consommer (et tu peux garder un domaine plus riche).
En gros, Le client veut afficher des libellés de tags, mais pas des objets métier complets.

Relance un test: `mvn -f api/pom.xml test`

Si tout est OK, on va maintenant attaquer l'étape suivante (MapStruct DTO ↔ Domaine), en commençant par déplacer le contrôleur dans `fr.myclilius.web.controller` et en lui faisant consommer `CreateWikiPageRequest` + `@Valid`.

Petit récap avant :

Jusqu'ici, on a créé les formulaires officiels (DTO).

A l'étape suivante, on réorganise le guichet et on le branche sur ces formulaires. Et pour faire cela, on doit :

1. Déplacer le contrôleur dans web.controller

C'est comme déplacer l'agent d'accueil dans le bon bureau "Guichet" du bâtiment, pour que tout le monde sache : "ici, c'est l'entrée officielle des dossiers".

2. Le contrôleur consomme CreateWikiPageRequest + @Valid

C'est comme dire au guichet : "désormais, tu n'acceptes que le formulaire officiel, et tu contrôles automatiquement qu'il est bien rempli avant de transmettre au service instructeur".

3. MapStruct ensuite (DTO ↔ Domaine)

C'est comme fournir au guichet une procédure de transcription automatique c'est-à-dire qu'au lieu de recopier à la main les champs du formulaire vers le dossier interne, on automatise cette transcription proprement et sans erreurs.

4. MapStruct : DTO ↔ Domaine (couche web)

Rappel d'intention :

Le contrôleur est un adaptateur HTTP : il reçoit DTO, appelle le service métier, renvoie DTO.

En gros, le contrôleur = le guichet.

Il reçoit un formulaire officiel (DTO), puis il transmet un dossier interne au service instructeur (Service), et renvoie une "fiche récapitulative" au citoyen (DTO de réponse).

MapStruct = la procédure interne de transcription : au lieu de recopier à la main chaque champ du formulaire vers le dossier interne, on génère un agent qui le fait correctement et toujours de la même manière.

Avant de créer le mapper : point de vigilance (important)

Ton mapper MapStruct doit produire/consommer des classes qui existent réellement :

- DTO : CreateWikiPageRequest, TagRequest, WikiPageResponse (OK, on vient de les créer)
- Domaine : WikiPage, Tag (chez toi, ils existent déjà, mais leur forme exacte compte)

MapStruct ne "devine" pas : il mappe soit par noms identiques, soit via règles explicites.

4.1 Mapper DTO → Domaine et Domaine → DTO

Fichier : api/src/main/java/fr/mycellius/web/mapper/WikiPageDtoMapper.java

```
package fr.mycellius.web.mapper;

import fr.mycellius.domain.Tag;
import fr.mycellius.domain.WikiPage;
import fr.mycellius.web.dto.CreateWikiPageRequest;
import fr.mycellius.web.dto.TagRequest;
import fr.mycellius.web.dto.WikiPageResponse;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper(componentModel = "spring")
public interface WikiPageDtoMapper {

    // DTO -> Domain (MapStruct mappe id/title/content automatiquement si les noms
    correspondent)
    WikiPage toDomain(CreateWikiPageRequest request);

    // DTO -> Domain (Tag immuable : on construit un nouvel objet)
    default Tag toDomain(TagRequest request) {
        if (request == null) return null;
    }
}
```

```

        return new Tag(request.name());
    }

    // Domain -> DTO
    // Pour l'instant on ignore createdAt si ton domaine ne l'a pas encore
    @Mapping(target = "createdAt", ignore = true)
    WikiPageResponse toResponse(WikiPage page);

    // Conversion métier -> front-friendly : Tag -> String
    default String map(Tag tag) {
        if (tag == null) return null;
        return tag.getValue();
    }
}

```

=> Relance le test : `mvn -f api/pom.xml test`

Ce fichier `WikiPageDtoMapper` est un élément central de la séance 6, parce qu'il matérialise la séparation "HTTP (DTO) ↔ Métier (Domaine)" et automatise la conversion entre ces deux mondes.

L'intérêt global du fichier (à quoi il sert vraiment)

Ce mapper sert à éviter que ton contrôleur fasse du "recopiage manuel" de champs.

Avant (sans mapper), le contrôleur ressemble souvent à ça :

- je lis `request.id()`, `request.title()`, etc.
- je construis un `WikiPage`
- puis je reconstruis un DTO de réponse

=> C'est répétitif, fragile, et ça pollue le contrôleur (qui devrait juste être un adaptateur HTTP).

Avec ce fichier :

le contrôleur reste "propre" : il reçoit un DTO, appelle le service, renvoie un DTO

la conversion DTO↔Domaine est regroupée dans un seul endroit, facile à maintenir

Analogie "dossier administratif"

- DTO = formulaire officiel rempli par le citoyen
- Domaine = dossier interne de l'administration

=> Le mapper = la procédure de transcription officielle entre le formulaire et le dossier.

Notion importante : MapStruct et la génération de code

```
@Mapper(componentModel = "spring")
```

Ce que ça veut dire :

MapStruct va générer automatiquement une classe concrète (ex : WikiPageDtoMapperImpl) à la compilation.

componentModel="spring" demande à MapStruct de générer cette classe comme un “bean” Spring, pour que tu puisses l’injecter dans ton contrôleur.

Analogie

C’est comme si tu écrivais la procédure “comment recopier un formulaire vers un dossier interne”, et qu’un outil génère automatiquement un agent qui applique cette procédure, et l’ajoute à l’organigramme officiel (Spring).

Notion importante : mapping automatique par convention

```
WikiPage toDomain(CreateWikiPageRequest request);
```

Tu n’écris aucune règle ici, pourtant ça mappe.

Pourquoi ?

MapStruct applique une convention :

- si le DTO a id, title, content
- et si le domaine a aussi id, title, content

=> alors MapStruct copie automatiquement ces champs.

Ce que vous devez retenir :

Si les noms correspondent, MapStruct fait le travail sans que tu écrives du code de conversion.

Notion importante : objets immuables et méthodes “default”

```
default Tag toDomain(TagRequest request) { ... }
```

Pourquoi tu as dû le faire “à la main” :

Ton Tag est immuable (pas de setter). MapStruct ne peut pas faire :

“je crée Tag vide”

“je fais setValue(...)”

=> Parce que ces méthodes n’existent pas.

Donc tu fournis une règle explicite :

“pour transformer un TagRequest en Tag, construis un Tag avec le name”

C’est un point pédagogique majeur :

MapStruct marche très bien avec des objets immuables, mais parfois il faut lui expliquer comment instancier l’objet.

Analogie

Au guichet, certains dossiers internes ne sont pas modifiables une fois créés : on ne “remplit pas des cases”, on “crée un nouveau dossier complet”. La méthode default décrit exactement cette procédure.

Notion importante : @Mapping(... ignore = true) et champ “technique”

@Mapping(target = "createdAt", ignore = true)

Ça veut dire :

- le DTO de réponse a un champ createdAt
- mais ton domaine n’a pas encore cette donnée (ou ne la fournit pas)
- => Donc on dit à MapStruct : “ne cherche pas à le remplir, laisse-le vide/à null”.

Pédagogiquement :

- certains champs sont produits par l’administration (DB/service), pas par le formulaire
- donc ils ne sont pas toujours disponibles au même moment dans toutes les couches

Notion importante : transformer un type métier en type “front-friendly”

```
default String map(Tag tag) { return tag.getValue(); }
```

C’est une règle de conversion de type :

- côté métier, un tag est un objet Tag
- côté API (front), un tag est une simple chaîne

MapStruct voit :

- dans WikiPageResponse, tags est une List<String>
- dans le domaine, tags est une List<Tag> et il comprend : “si je sais convertir un Tag en String, je sais convertir une liste de Tag en liste de String”.

Donc la méthode map(Tag) est très stratégique : elle permet le mapping automatique des collections.

Analogie

Dans le dossier interne, tu as des “objets officiels” (Tag avec structure).

Dans la réponse au citoyen, tu veux une info simple à lire (libellé).

Cette méthode est la règle de simplification.

Relations fonctionnelles entre les fichiers (qui utilise quoi)

- CreateWikiPageRequest (DTO entrée) : reçu par le contrôleur
- WikiPageDtoMapper.toDomain(...) : transforme ce DTO en WikiPage (domaine)
- WikiService : applique la logique métier sur le domaine
- WikiPageDtoMapper.toResponse(...) : transforme le domaine en WikiPageResponse (DTO sortie)
- WikiPageResponse : renvoyé au client

Le contrôleur devient donc un simple “passe-plat” HTTP :

DTO in → mapper → domaine → service → domaine → mapper → DTO out.

4.2 — Déplacer le contrôleur dans fr.mycellius.web.controller

Action 1 — Déplacer le fichier

Dans IntelliJ, trouve ton fichier actuel :

api/src/main/java/fr/mycellius/web/WikiPageController.java

Déplace-le vers :

api/src/main/java/fr/mycellius/web/controller/WikiPageController.java

(Important : utilise “Refactor > Move” si possible, pour éviter des oublis.)

Action 2 — Modifier le package en haut du fichier

Tout en haut, remplace si ce n'est pas déjà fait :

package fr.mycellius.web;

par :

package fr.mycellius.web.controller;

Action 3 — Vérifier les imports (rien d'autre)

Normalement IntelliJ corrige automatiquement. Si tu as des imports cassés, tu les laisses tels quels et tu relances une compilation : on corrigera proprement ensuite.

Test (obligatoire)

Depuis la racine : `mvn -f api/pom.xml test`

Résultat attendu : BUILD SUCCESS

Et test fonctionnel rapide (pour s'assurer qu'on n'a rien cassé)

Dans Postman :

GET <http://localhost:8080/api/pages/PAGE-DB-001> doit encore renvoyer 200.

attendu :

build OK et GET témoin OK

The screenshot shows the Postman interface for a GET request to `http://localhost:8080/api/pages/PAGE-DB-001`. The response status is 200 OK. The response body is in JSON format, displaying the following data:

```
{
  "id": "PAGE-DB-001",
  "title": "Page témoin avant refactor",
  "content": "Cette page doit rester lisible après la séance 6."
}
```

On récapitule un peu :

- Précondition de test validée :

tu as créé une page témoin en base (PAGE-DB-001) via POST `/api/pages`, puis tu as vérifié immédiatement qu'elle est relisible via GET `/api/pages/PAGE-DB-001`. Cette page sert de "preuve" avant/après refactor.

- Arborescence cible créée :

tu as préparé les packages pour séparer proprement HTTP (web), métier (domain), et base (persistence).

- DTO + Validation Jakarta créés :

> TagRequest (entrée) avec contraintes @NotBlank + @Size

> CreateWikiPageRequest (entrée) avec contraintes sur id/title/content + @Valid sur la liste de tags

> WikiPageResponse (sortie) avec tags en List<String> et createdAt

- MapStruct web en place :

WikiPageDtoMapper compile et les tests passent.

- Contrôleur déplacé dans fr.mycellius.web.controller (organisation/lecture du projet), sans changer son comportement.

Ce qu'on va faire ensuite (succinct, mais très clair)

A) Faire consommer CreateWikiPageRequest + @Valid dans le contrôleur

Objectif :

le contrôleur n'accepte plus "n'importe quel JSON" ni un ancien record sans validation, il accepte explicitement le formulaire officiel.

Concrètement, on va modifier la signature du POST pour passer de :

createPage(@RequestBody CreatePageRequest ...)

à :

createPage(@Valid @RequestBody CreateWikiPageRequest ...)

Effet immédiat :

si on envoie title="" ou un id au mauvais format, Spring déclenche automatiquement un 400 (le guichet refuse le dossier incomplet).

B) Remplacer le mapping manuel par WikiPageDtoMapper

Objectif :

sortir la logique de conversion du contrôleur.

Concrètement, au lieu d'écrire dans le contrôleur :

"je construis un WikiPage à la main"

"je reconvertis un WikiPage en WikiPageResponse à la main"

on fera :

WikiPage page = mapper.toDomain(request)

WikiPageResponse response = mapper.toResponse(page)

Avantages :

- contrôleur plus lisible (rôle d'adaptateur HTTP uniquement)
- conversions centralisées (un seul endroit à maintenir)
- cohérence garantie (mêmes règles de mapping partout)

4.3 - Le contrôleur consomme CreateWikiPageRequest + @Valid

1. Modifier le contrôleur (uniquement la méthode POST)

Avant de continuer, voici une petite analogie...

Tu es au guichet. Tu dis : "je veux accepter le formulaire officiel CreateWikiPageRequest et appliquer la règle de contrôle @Valid".

Mais si ton agent (le fichier Java) n'a pas ces documents dans son classeur (imports), il ne sait pas de quoi tu parles.

Fichier : `api/src/main/java/fr/mycellius/web/controller/WikiPageController.java`

A. Ajoute les imports (si absents) :

```
import fr.mycellius.web.dto.CreateWikiPageRequest;
import jakarta.validation.Valid;
```

B. Modifie la signature de la méthode POST pour qu'elle devienne :

```
@PostMapping
public ResponseEntity<WikiPage> createPage(@Valid @RequestBody CreateWikiPageRequest request) {
    List<Tag> domainTags = toDomainTags(request.tags());
    WikiPage page = wikiService.createPage(
        request.id(),
        request.title(),
        request.content(),
        domainTags
    );
    return ResponseEntity.status(HttpStatus.CREATED).body(page);
}
```

et rajoute en bas du controller, cette méthode utilitaire :

```
private List<Tag> toDomainTags(List<TagRequest> tags) {
    if (tags == null) return List.of();
    return tags.stream()
        .map(TagRequest::name)
        .map(Tag::new)
        .toList();
}
```

Lance le test : `mvn -f api/pom.xml test`

Important : à ce stade, il est possible que `request.tags()` ne soit pas exactement du type attendu par `wikiService.createPage(...)` (parce que désormais `tags` est une `List<TagRequest>` et ton service attend peut-être `List<Tag>`).

Si ça ne compile pas, c'est normal : on n'a pas encore remplacé par `MapStruct` et on n'a pas encore ajusté le mapping. Dans ce cas, on fera un micro-ajustement (très local) juste pour compiler, puis on basculera vers `MapStruct` juste après.

Tu risques d'avoir des bugs à cause de la conf des Tags, il faudra modifier le fichier en collant le code ci dessous dans `WikiPageDtoMapper` :

```
package fr.mycellius.web.mapper;

import fr.mycellius.domain.Tag;
import fr.mycellius.domain.WikiPage;
import fr.mycellius.web.dto.CreateWikiPageRequest;
import fr.mycellius.web.dto.TagRequest;
import fr.mycellius.web.dto.WikiPageResponse;
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper(componentModel = "spring")
public interface WikiPageDtoMapper {

    // DTO -> Domain
    // MapStruct mappe id/title/content automatiquement si les noms correspondent
    WikiPage toDomain(CreateWikiPageRequest request);

    // DTO -> Domain (Tag immuable : on construit un nouvel objet)
    default Tag toDomain(TagRequest request) {
        if (request == null) return null;
        return new Tag(request.name());
    }

    // Domain -> DTO
    // - createdAt ignoré tant que le domaine ne le porte pas
    // - tags renvoyés vides tant que WikiPage n'expose pas une liste de tags
    @Mapping(target = "createdAt", ignore = true)
    @Mapping(target = "tags", expression = "java(java.util.List.of())")
    WikiPageResponse toResponse(WikiPage page);
}
```

Test compilation (obligatoire)

Lance : `mvn -f api/pom.xml test`

Résultat attendu : BUILD SUCCESS.

Test validation (obligatoire, Postman)

POST <http://localhost:8080/api/pages> avec :

```
{
  "id": "BAD",
  "title": "",
  "content": "test",
  "tags": []
}
```

Résultat attendu : 400 Bad Request

The screenshot displays the Postman application interface. On the left, a sidebar shows a collection named 'Séance 6 - page témoin' with three requests: '01 - POST create witness PAGE-DB-001', '02 - GET witness PAGE-DB-001', and '03 - POST create witness PAGE-BAD'. The third request is selected. The main panel shows the details of this request, which is a POST to 'http://localhost:8080/api/pages'. The request body is a JSON object:

```
{
  "id": "BAD",
  "title": "",
  "content": "test",
  "tags": []
}
```

. Below the request details, the response is shown as a '400 Bad Request' status. The response body is a JSON object:

```
{
  "timestamp": "2026-01-27T09:59:51.663+00:00",
  "status": 400,
  "error": "Bad Request",
  "path": "/api/pages"
}
```

4.4 - MapStruct ensuite (DTO ↔ Domaine)

On reprend l'analogie pour comprendre où on va :

Analogie “dossier administratif” version ultra concrète : formulaire papier → logiciel interne

Imagine une administration où :

1. L'utilisateur remplit un formulaire papier (DTO)

Ex : “Demande de création de page”

- id
- titre
- contenu
- tags (une liste écrite à la main, ligne par ligne)

2. À l'accueil (contrôleur), on ne tape pas tout dans le logiciel :

L'accueil ne fait pas la saisie complète, sinon :

- c'est long
- ce n'est pas son rôle
- il y a beaucoup d'erreurs possibles

Son rôle est surtout :

- vérifier que le formulaire papier est valide (cases obligatoires remplies) → @Valid
- transmettre le formulaire à la personne qui fait la saisie

3. La “personne de saisie” (mapper / MapStruct) convertit le papier en format logiciel (domaine):

Le logiciel interne (domaine) n'accepte pas un “papier” : il veut des objets structurés.

Exemple TRÈS concret :

- Sur le papier : tags = [{"name":"dev"}, {"name":"java"}] (DTO)
- Dans le logiciel : tags = [Tag("dev"), Tag("java")] (domaine)

Ce que faisait votre contrôleur jusqu'ici...

Il faisait lui-même la saisie des tags :

toDomainTags(...) = “je prends chaque ligne de tags du papier et je retape en Tag(...)”

Ce qu'on veut maintenant...

On veut que le contrôleur arrête de faire cette saisie, et qu'il délègue à la “personne de saisie” officielle : le mapper.

=> Donc on déplace ce bloc, du contrôleur (guichet) vers WikiPageDtoMapper (service de saisie)

4. On obtient la Règle simple :

- Le contrôleur = contrôle + transmission
- Le mapper = conversion / saisie (papier → logiciel)
- Le service = traitement métier

4.4.1 : ajouter toDomainTags(...) dans le mapper

Rappel d'intention (analogie)

- Formulaire papier (DTO) : l'utilisateur écrit les tags ligne par ligne :

```
[{"name":"dev"}, {"name":"java"}]
```

- Logiciel interne (domaine) : l'administration veut des objets propres :

```
[Tag("dev"), Tag("java")]
```

Aujourd'hui, votre accueil (contrôleur) fait encore cette saisie "à la main" avec une méthode toDomainTags(...).

En 4.4.1, on veut que ce soit la personne de saisie officielle (le mapper) qui fasse cette conversion.

Dans :

api/src/main/java/fr/mycellius/web/mapper/WikiPageDtoMapper.java

Dans ce fichier, vous avez :

```
default Tag toDomain(TagRequest request) {  
    if (request == null) return null;  
    return new Tag(request.name());  
}
```

On va ajouter juste après la méthode toDomainTags():

```
default java.util.List<Tag> toDomainTags(java.util.List<TagRequest> tags) {  
    if (tags == null) return java.util.List.of();  
    return tags.stream()  
        .map(this::toDomain)           // TagRequest -> Tag  
        .filter(java.util.Objects::nonNull) // sécurité  
        .toList();  
}
```

Puis : mvn -f api/pom.xml test

Ce que fait cette méthode :

- Elle prend la liste papier : List<TagRequest>
- Elle transforme chaque ligne en objet interne : new Tag(request.name())
- Elle renvoie la liste interne : List<Tag>

À ce stade, on n'a pas encore touché au contrôleur. On a simplement donné au "service de saisie" (mapper) la capacité de saisir une liste entière de tags.

4.4.2 modifier le contrôleur

Avant : l'accueil retapait les tags dans le logiciel (méthode toDomainTags dans le contrôleur).

Maintenant : l'accueil donne la feuille "tags" à la personne de saisie (mapper) qui sait convertir proprement.

On va maintenant supprimer la méthode toDomainTags(...) du contrôleur et la remplacer par mapper.toDomainTags(request.tags()).

Ne corrige rien, tu vas avoir des alertes et des warnings mais ils vont disparaître au fur et à mesure en suivant la procédure....

Dans le fichier : api/src/main/java/fr/mycellius/web/controller/WikiPageController.java

1. Ajouter l'injection du mapper

En haut, ajoute l'import :

```
import fr.mycellius.web.mapper.WikiPageDtoMapper;
```

Puis dans la classe, ajoute un champ :

```
private final WikiPageDtoMapper mapper;
```

Et modifie le constructeur pour recevoir le mapper :

```
public WikiPageController(WikiService wikiService, WikiPageDtoMapper mapper) {  
    this.wikiService = wikiService;  
    this.mapper = mapper;  
}
```

2. Remplacer l'appel au mapping manuel dans le POST

Dans ta méthode createPage, remplace :

```
List<Tag> domainTags = toDomainTags(request.tags());
```

par :

```
List<Tag> domainTags = mapper.toDomainTags(request.tags());
```

3. Supprimer la méthode utilitaire du contrôleur

Supprime entièrement cette méthode du contrôleur :

```
private List<Tag> toDomainTags(List<TagRequest> tags) {  
    if (tags == null) return List.of();  
    return tags.stream()  
        .map(TagRequest::name)  
        .map(Tag::new)  
        .toList();  
}
```

4. Nettoyer les imports devenus inutiles

Après suppression, tu n'as normalement plus besoin de :

```
import fr.mycellius.web.dto.TagRequest;
```

=> Supprime-le si IntelliJ le marque "unused".

À ce stade, ton contrôleur doit compiler avec :

- WikiService
- WikiPageDtoMapper
- CreateWikiPageRequest
- @Valid
- Tag (car List<Tag> est encore utilisé)

On teste : `mvn -f api/pom.xml test`

=> Résultat attendu : BUILD SUCCESS.

4.4.3 — Modifier les types de retour dans le contrôleur

Ici, on ne renvoie plus l'objet interne WikiPage (dossier interne), on renvoie un WikiPageResponse (récépissé).

On ne touche pas aux tags en sortie : votre mapper renvoie déjà tags = [] volontairement

En gros :

- WikiPage = dossier interne du logiciel (pas destiné au public).
- WikiPageResponse = réception remis à l'utilisateur (format stable, lisible).

Dans le fichier : `api/src/main/java/fr/mycellius/web/controller/WikiPageController.java`

1. Imports à ajouter (ajoutez l'import si absent) :

```
import fr.mycellius.web.dto.WikiPageResponse;
```


2. POST : retourner WikiPageResponse au lieu de WikiPage

Remplacez la signature :

```
public ResponseEntity<WikiPage> createPage(@Valid @RequestBody CreateWikiPageRequest request)
```

par :

```
public ResponseEntity<WikiPageResponse> createPage(@Valid @RequestBody CreateWikiPageRequest request)
```

Puis remplacez le return :

Avant :

```
return ResponseEntity.status(HttpStatus.CREATED).body(page);
```

Après :

```
return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toResponse(page));
```

3. GET by id : retourner WikiPageResponse

Remplacez :

```
public WikiPage getPageById(@PathVariable String id) {  
    return wikiService.getPageById(id);  
}
```

par :

```
public WikiPageResponse getPageById(@PathVariable String id) {  
    return mapper.toResponse(wikiService.getPageById(id));  
}
```

4. GET search : retourner List<WikiPageResponse>

Remplacez :

```
public List<WikiPage> searchByTitle(@RequestParam("title") String fragment) {  
    return wikiService.searchByTitle(fragment);  
}
```

par :

```
public List<WikiPageResponse> searchByTitle(@RequestParam("title") String fragment) {  
    return wikiService.searchByTitle(fragment).stream()  
        .map(mapper::toResponse)  
        .toList();  
}
```

5. Nettoyage imports

Après ces changements, l'import `fr.myclius.domain.WikiPage` peut devenir inutile dans le contrôleur (selon votre code). Si IntelliJ le marque "unused", supprimez-le.

6. Test : `mvn -f api/pom.xml test` => Attendu : BUILD SUCCESS.

7. Test fonctionnel rapide (Postman)

- GET `/api/pages/{id}` doit renvoyer un JSON au format `WikiPageResponse` en remplaçant `{id}` par `PAGE-DB-001` ou ce que tu as en POST dans postman.

Tu auras un truc comme ça :

```
{
  "id": "PAGE-DB-001",
  "title": "Page témoin avant refactor",
  "content": "Cette page doit rester lisible après la séance 6."
}
```

- Mais, nous ici, on aimerait voir afficher => tags qui sera [] et createdAt qui sera null.
Pour cela, va dans ton `application.properties` et rajoute à la fin du fichier :

```
spring.jackson.default-property-inclusion=ALWAYS
```

Jackson est le convertisseur Java "objet Java → JSON". Et là, il dit : "Dans le JSON, écris tous les champs de l'objet, même si leur valeur est null ou vide"

- Refais ta requête sur Postman et tu devrais avoir une réponse "complète" :

```
{
  "id": "PAGE-DB-001",
  "title": "Page témoin avant refactor",
  "content": "Cette page doit rester lisible après la séance 6.",
  "tags": [],
  "createdAt": null
}
```

5. Modèle DB tags : migrations Flyway

En séance 5, tu as une seule table `wiki_page` et un index `title` en V2 (optionnel).

On va maintenant ajouter TAG + table d'association.

5.1 Ajouter la migration V3 (tables tag + table d'association)

On prépare la base MySQL pour gérer des tags "comme au resto" : une page peut avoir plusieurs tags, et un tag peut servir à plusieurs pages.

Ça demande deux choses côté base :

- Une table `tag` = la liste des "ingrédients" uniques (dev, onboarding, java...)
- Une table `wiki_page_tag` = le "bon de composition" qui dit : "cette assiette (page) contient ces ingrédients (tags)"

Analogie restaurant :

- `wiki_page` = une assiette / un plat (ex : "PAGE-001")
- `tag` = un ingrédient standardisé (ex : "dev", "onboarding")
- `wiki_page_tag` = la fiche "composition du plat" : quels ingrédients sont dans quel plat

Pourquoi c'est une table d'association

Parce qu'un plat peut avoir plusieurs ingrédients, et un ingrédient peut être utilisé dans plusieurs plats : c'est une relation **Many-to-Many**.

Crée le Fichier : `api/src/main/resources/db/migration/V3__create_tag_tables.sql`

```
CREATE TABLE IF NOT EXISTS tag (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  value VARCHAR(30) NOT NULL UNIQUE  
);  
  
CREATE TABLE IF NOT EXISTS wiki_page_tag (  
  page_id VARCHAR(64) NOT NULL,  
  tag_id BIGINT NOT NULL,  
  PRIMARY KEY (page_id, tag_id),  
  CONSTRAINT fk_wpt_page FOREIGN KEY (page_id) REFERENCES wiki_page(id),  
  CONSTRAINT fk_wpt_tag FOREIGN KEY (tag_id) REFERENCES tag(id)  
);
```

Pourquoi UNIQUE sur tag.value :

Sans ça, tu peux te retrouver avec 10 ingrédients différents pour “le même truc” :

=> “dev”

=> “Dev”

=> “DEV”

Résultat : la cuisine ne sait plus quoi utiliser, le front ne sait plus quoi proposer.

Le **UNIQUE** impose : un seul ingrédient officiel par nom.

Tu viens d’ajouter une nouvelle “étagère d’ingrédients” (tag) et une “fiche recette” (wiki_page_tag). Maintenant on vérifie que le restaurant (Flyway) a bien installé ces éléments dans la cuisine (MySQL).

1. Pour vérifier cela, on redémarre l'API depuis la racine du projet pour déclencher FlyWay:

```
mvn -f api/pom.xml spring-boot:run
```

2. Vérifie les logs Flyway au démarrage

Tu dois voir une ligne indiquant que Flyway migre vers la version 3, ou au minimum qu’il a “migrated/validated” et que le schéma est à jour avec version 3.

Output possible :

“Migrating schema ... to version 3”

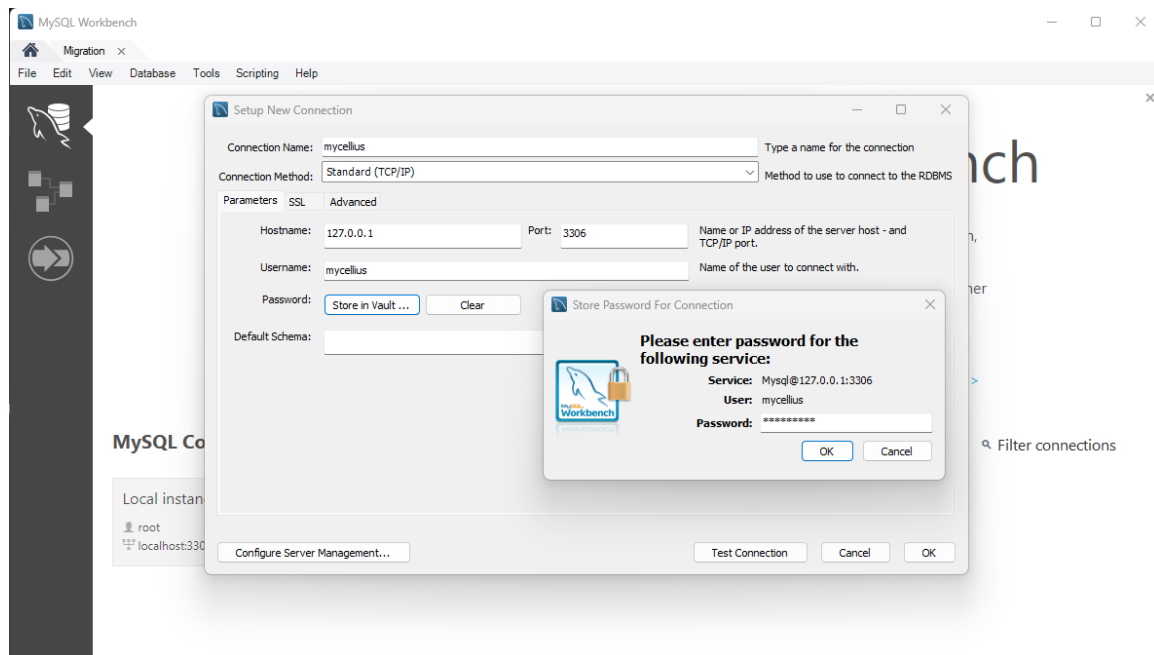
puis “Successfully applied ... V3__create_tag_tables.sql”

et “Current version of schema ... : 3” (ou “Schema is up to date” avec version 3)

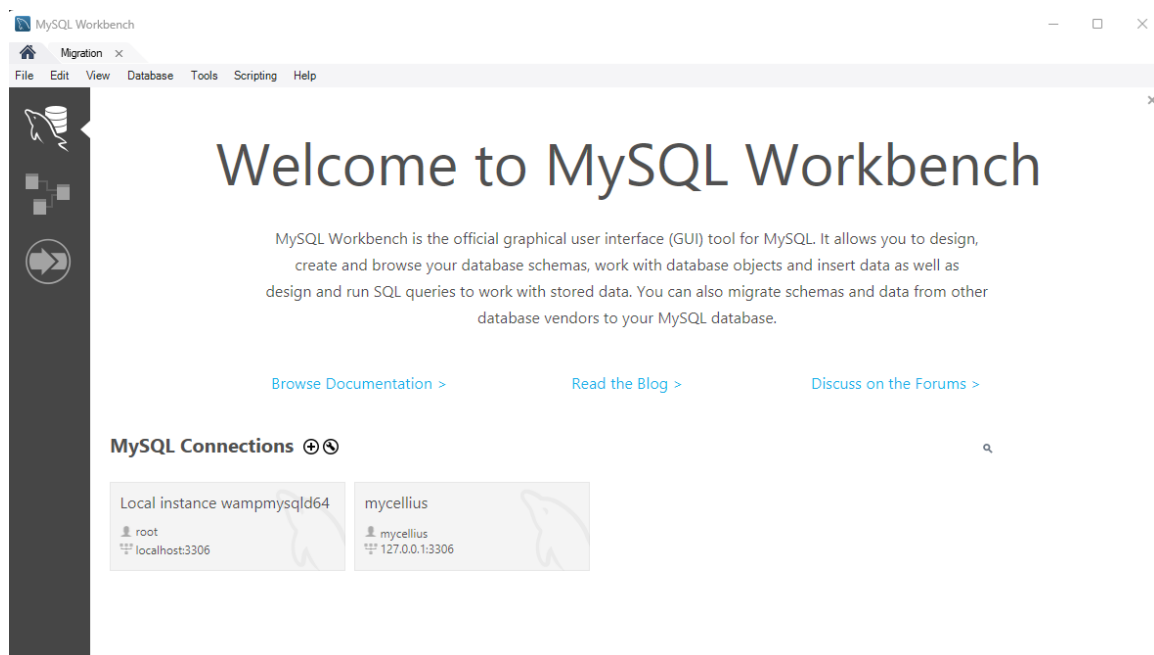
```
: Successfully validated 3 migrations (execution time 00:00.037s)
: Current version of schema 'mycellius': 2
: Migrating schema 'mycellius' to version '3 - create tag tables'
: Successfully applied 1 migration to schema 'mycellius', now at version v3 (execution time 00:00.432s)
```

3. Vérification en base MySQL avec MySQL Workbench :

- Clique sur le + à droite de MySQL Connections pour rajouter une connexion et saisis les informations de la bdd => +



Tu dois obtenir ceci après :



- Dans MySQL Workbench, double-clique ta connexion (celle avec user mycellius).
- Une fois connecté, ouvre un onglet SQL (souvent "SQL +" ou "Create a new SQL tab").

- Copie/colle ces 2 lignes puis exécute (⚡ ou Ctrl+Enter) :

```
USE mycellius;
```

```
SHOW TABLES;
```

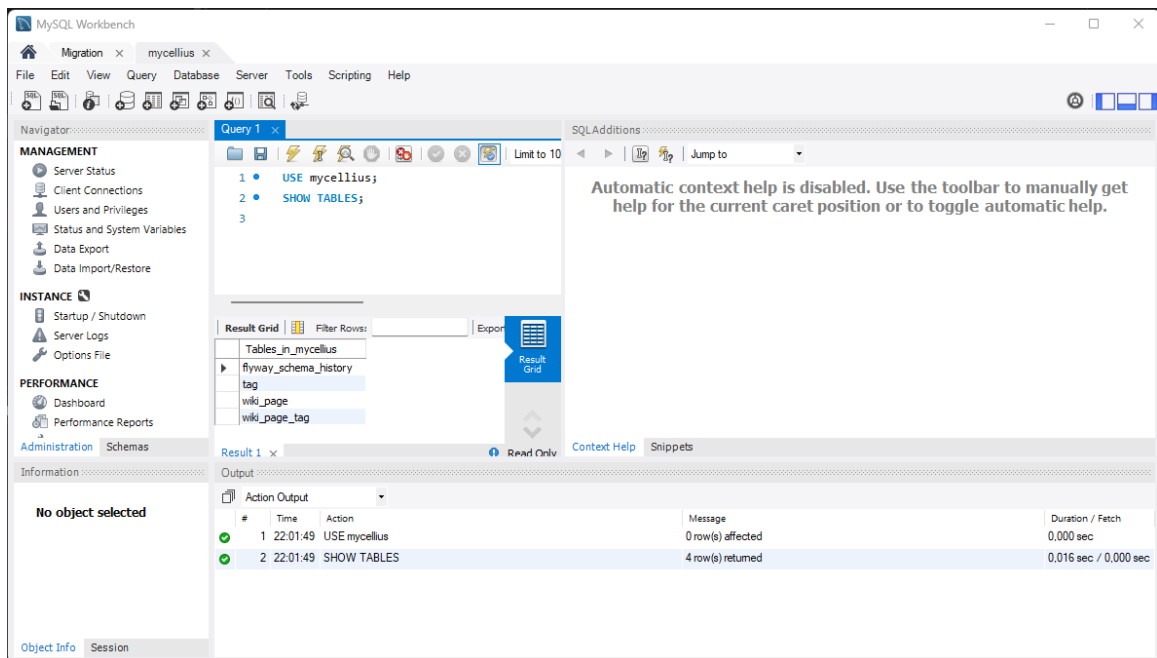
Résultat attendu :

- wiki_page

- tag

- wiki_page_tag

(et très souvent aussi flyway_schema_history, c'est normal)



5.2 Migration V4 : index utiles (prise en compte de V2 qui a déjà l'index title.)

Imagine toi dans une bibliothèque, il te faut :

1. Les tables MySQL = des étagères / rayons

- wiki_page = le rayon "Livres" (chaque page = un livre).
- tag = le fichier "Mots-clés" (chaque tag = une fiche mot-clé).
- wiki_page_tag = le registre "Livre ↔ mots-clés" (quel livre est lié à quels mots-clés).

2. Un index = le catalogue / l'index alphabétique

Sans index sur tag.value :

- tu as un tiroir avec toutes les fiches mots-clés mélangées
- pour trouver la fiche "dev", tu dois feuilleter toutes les fiches une par une.

Avec un index sur tag.value :

- tu as un catalogue alphabétique des mots-clés
- tu vas directement à "D → dev", sans parcourir le reste.

3. Pourquoi on ne refait pas l'index sur wiki_page.title

Tu as déjà (en V2) un catalogue pour retrouver les livres par titre (wiki_page.title).

Donc refaire un index sur wiki_page.title en V4, c'est comme créer un second catalogue identique du même rayon et résultat, soit MySQL refuse ("catalogue déjà existant"), soit c'est redondant.

Donc pour V4 (dans notre cas) :

- on ajoute seulement le catalogue manquant : idx_tag_value sur tag(value)
- on ne touche pas au titre des pages car V2 l'a déjà couvert.

Concrètement, crée ton fichier V4 doit contenir uniquement :

Fichier : api/src/main/resources/db/migration/V4__add_indexes_for_search.sql

```
CREATE INDEX idx_tag_value ON tag(value);
```

Check DB :

Relance l'API (mvn -f api/pom.xml spring-boot:run) et vérifie que Flyway joue bien V3/V4 comme tout à l'heure.

```
Database: jdbc:mysql://localhost:3306/mycellius?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC (MySQL 8.0)
Successfully validated 4 migrations (execution time 00:00.043s)
Current version of schema 'mycellius': 3
Migrating schema 'mycellius' to version "4 - add indexes for search"
Successfully applied 1 migration to schema 'mycellius', now at version v4 (execution time 00:00.113s)
```

6. Entités JPA : TagEntity + relation ManyToMany

6.1 Création de l'entité JPA TagEntity

Analogie "bibliothèque" :

La table tag, c'est une nouvelle étagère dédiée aux "mots-clés".

TagEntity, c'est la fiche cartonnée (la "notice") qui décrit comment un tag est rangé sur cette étagère (colonnes, contraintes).

On crée la fiche AVANT de relier les livres aux mots-clés (la relation ManyToMany viendra en 6.2).

Maintenant, crée le fichier :

=> api/src/main/java/fr/mycellius/persistence/entity/TagEntity.java

```
package fr.mycellius.persistence.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "tag", uniqueConstraints = @UniqueConstraint(name = "uk_tag_value", columnNames = "value"))
public class TagEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "value", nullable = false, length = 30)
    private String value;

    public TagEntity() {}

    public TagEntity(String value) {
        this.value = value;
    }

    public Long getId() { return id; }
    public String getValue() { return value; }
}
```

Check compilation : mvn -f api/pom.xml test => On attend : **BUILD SUCCESS** (évidemment !!!)

6.2 Ajout de tags dans WikiPageEntity

On va ajouter la relation tags dans WikiPageEntity (sans encore toucher au reste).

Que se passe-t-il ici ?

- wiki_page, c'est l'étagère des livres.
- tag, c'est l'étagère des mots-clés.
- wiki_page_tag, c'est la "fiche de liaison" qui dit : "ce livre est associé à ces mots-clés".

=> Dans le code, la relation ManyToMany, c'est exactement cette fiche de liaison.

On va s'occuper maintenant de (uniquement WikiPageEntity qui n'est pas à la bonne place dans l'arborescence du projet). Refactor > Move to :

api/src/main/java/fr/mycellius/persistence/entity/WikiPageEntity.java

Ton ancien code était une version "simple" (id/title/content). Maintenant ta base a évolué : created_at est NOT NULL et tu as une relation ManyToMany via wiki_page_tag. L'entité doit refléter ça, sinon tu vas te battre contre Hibernate.

Hibernate, c'est "le **traducteur**" entre tes objets Java (les Entity JPA comme WikiPageEntity, TagEntity) et la base MySQL.

Concrètement :

tu manipules des objets en Java (entity.setTitle(...), repository.save(entity)), Hibernate génère et exécute les requêtes SQL à ta place (INSERT, UPDATE, SELECT, jointures...), il "remplit" aussi tes objets quand il lit la base (il reconstruit une WikiPageEntity à partir d'une ligne SQL).

Dans le projet : Spring Data JPA te donne les interfaces JpaRepository, et derrière, c'est Hibernate qui fait le vrai boulot de persistance.

Ouvre le Fichier et colle le code suivant :

```
package fr.mycellius.persistence.entity;

import jakarta.persistence.*;

import java.time.Instant;
import java.util.HashSet;
import java.util.Set;

@Entity
@SuppressWarnings("unused")
@Table(name = "wiki_page")
public class WikiPageEntity {

    @Id
    @Column(name = "id", length = 64)
    private String id;
```

```

@Column(name = "title", nullable = false)
private String title;

@Column(name = "content", nullable = false, columnDefinition = "TEXT")
private String content;

@Column(name = "created_at", nullable = false)
private Instant createdAt;

@ManyToMany
@JoinTable(
    name = "wiki_page_tag",
    joinColumns = @JoinColumn(name = "page_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private Set<TagEntity> tags = new HashSet<>();

public WikiPageEntity() {
    // constructeur sans argument pour JPA
}

// Getters
public String getId() { return id; }
public String getTitle() { return title; }
public String getContent() { return content; }
public Instant getCreatedAt() { return createdAt; }
public Set<TagEntity> getTags() { return tags; }

// Setters
public void setId(String id) { this.id = id; }
public void setTitle(String title) { this.title = title; }
public void setContent(String content) { this.content = content; }
public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
public void setTags(Set<TagEntity> tags) { this.tags = tags; }
}

```

Tu vas avoir des bugs mais c'est normal vu que notre code a évolué.
Allez, on regarde ça !

On a remplacé WikiPageEntity par une version “complète” (avec createdAt + tags) et on a supprimé le constructeur (id, title, content). Or DatabaseWikiRepository l'utilise encore.

On adapte donc DatabaseWikiRepository pour utiliser le constructeur vide + setters.

Ouvre : `api/src/main/java/fr/mycellius/repository/DatabaseWikiRepository.java`

Repère la ligne qui ressemble à ça (celle qui plante) :

```
WikiPageEntity entity = new WikiPageEntity(page.getId(), page.getTitle(), page.getContent());
```

Remplace-la par :

```
WikiPageEntity entity = new WikiPageEntity();  
entity.setId(page.getId());  
entity.setTitle(page.getTitle());  
entity.setContent(page.getContent());
```

```
// created_at est NOT NULL en DB, donc on met une valeur  
entity.setCreatedAt(java.time.Instant.now());
```

Relance : mvn -f api/pom.xml test

7. Persistence : repositories JPA + MapStruct Domaine ↔ Entity + résolution tags

Ici, on reprend exactement le principe “WikiRepository = contrat / DatabaseWikiRepository = traducteur” décrit en séance 5.

La nouveauté : les tags imposent une règle “find or create”.

7.1 TagJpaRepository

Crée le fichier : `api/src/main/java/fr/mycellius/persistence/jpa/TagJpaRepository.java`

```
package fr.mycellius.persistence.jpa;

import fr.mycellius.persistence.entity.TagEntity;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface TagJpaRepository extends JpaRepository<TagEntity, Long> {
    Optional<TagEntity> findByValue(String value);
}
```

Lance : `mvn -f api/pom.xml test`

7.2 WikiPageJpaRepository (pagination + recherche)

Crée le fichier : `api/src/main/java/fr/mycellius/persistence/jpa/WikiPageJpaRepository.java`

```
package fr.mycellius.persistence.jpa;

import fr.mycellius.persistence.entity.WikiPageEntity;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;

public interface WikiPageJpaRepository extends JpaRepository<WikiPageEntity, String> {
    Page<WikiPageEntity> findByTitleContainingIgnoreCase(String title, Pageable
pageable);
}
```

Lance : `mvn -f api/pom.xml test`

7.3 Mapper Entity ↔ Domaine (MapStruct)

On a modifié des fichiers mais pas modifié l'objet métier qui est WikiPage (domaine) qui ne possède pas de propriété tags ni createdAt. Du coup MapStruct essaiera de remplir tags... mais il n'aura nulle part où le mettre.

On va donc corriger **WikiPage** avant.

Va dans le fichier : `api/src/main/java/fr/mycellius/domain/WikiPage.java`

Remplace le contenu existant par :

```
package fr.mycellius.domain;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

public class WikiPage {

    private String id;
    private String title;
    private String content;

    // ✅ Nouveau : tags (métier)
    private List<Tag> tags = new ArrayList<>();

    // ✅ Nouveau : createdAt (métier / utile pour tri + retour API plus tard)
    private Instant createdAt;

    public WikiPage() {
        // constructeur vide pour MapStruct (instanciation + setters)
    }

    public WikiPage(String id, String title, String content) {
        this(id, title, content, List.of(), null);
    }

    public WikiPage(String id, String title, String content, List<Tag> tags) {
        this(id, title, content, tags, null);
    }

    public WikiPage(String id, String title, String content, List<Tag> tags, Instant createdAt) {
        this.id = id;

        this.title = cleanTitle(title); // ✅ validation + trim
        this.content = content;
        this.tags = (tags == null) ? new ArrayList<>() : new ArrayList<>(tags);
        this.createdAt = createdAt;
    }
}
```

```
// ✅ Source de vérité : règles métier sur le titre
private static String cleanTitle(String title) {
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("Le titre est obligatoire");
    }
    return title.trim();
}

public String getId() { return id; }
public String getTitle() { return title; }
public String getContent() { return content; }

public List<Tag> getTags() { return tags; }
public Instant getCreatedAt() { return createdAt; }

public void setId(String id) { this.id = id; }

// ✅ MapStruct passe ici : on protège aussi ce chemin-là
public void setTitle(String title) { this.title = cleanTitle(title); }

public void setContent(String content) { this.content = content; }

public void setTags(List<Tag> tags) {
    this.tags = (tags == null) ? new ArrayList<>() : new ArrayList<>(tags);
}

public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
}
```

puis lance : `mvn -f api/pom.xml test`

Tu devrais avoir un bug car ton DTO d'entrée `CreateWikiPageRequest` n'a pas de champ `createdAt` alors que ton domaine `WikiPage` a maintenant `createdAt` et donc `MapStruct` est perdu car il ne sait pas quoi mettre dans le `createdAt`.

Dans `api/src/main/java/fr/mycellius/web/mapper/WikiPageDtoMapper.java`, modifie **UNIQUEMENT** la méthode `toDomain` comme ceci en remplaçant :

```
WikiPage toDomain(CreateWikiPageRequest request);
```

Par :

```
import org.mapstruct.Mapper;
import org.mapstruct.Mapping;

@Mapper(componentModel = "spring")
public interface WikiPageDtoMapper {

    @Mapping(target = "createdAt", ignore = true)
    WikiPage toDomain(CreateWikiPageRequest request);
}
```

```
// ... le reste inchangé  
}
```

puis lance : mvn -f api/pom.xml test

Crée le fichier : api/src/main/java/fr/mycellius/persistence/mapper/WikiPageEntityMapper.java

```
package fr.mycellius.persistence.mapper;  
  
import fr.mycellius.domain.Tag;  
import fr.mycellius.domain.WikiPage;  
import fr.mycellius.persistence.entity.TagEntity;  
import fr.mycellius.persistence.entity.WikiPageEntity;  
import org.mapstruct.Mapper;  
import org.mapstruct.Mapping;  
  
import java.util.List;  
import java.util.Set;  
  
@Mapper(componentModel = "spring")  
public interface WikiPageEntityMapper {  
  
    @Mapping(target = "tags", expression = "java(toDomainTags(entity.getTags()))")  
    WikiPage toDomain(WikiPageEntity entity);  
  
    default List<Tag> toDomainTags(Set<TagEntity> tags) {  
        if (tags == null) return List.of();  
        return tags.stream().map(t -> new Tag(t.getValue())).toList();  
    }  
  
    @Mapping(target = "tags", ignore = true)  
    WikiPageEntity toEntity(WikiPage page);  
}
```

Lance : mvn -f api/pom.xml test

Pourquoi ignore tags en Domain→Entity :

- parce que transformer une liste de tags en TagEntity nécessite d'aller en base (findByValue).
- MapStruct est un mapper, pas un accès DB. On garde donc la logique "find or create" dans l'adapter DB.

Analogie "bibliothèque" :

- WikiPageEntity = la "fiche catalogue" telle qu'elle est stockée en base.
- WikiPage (domaine) = la "fiche métier" utilisée par le personnel (service).
- Le mapper = le "bibliothécaire" qui recopie/convertit d'un format à l'autre.

- On ignore tags en Domain→Entity car pour convertir correctement il faut aller “consulter l’inventaire” (DB) pour faire le find or create, et ça ce sera le job de DatabaseWikiRepository, pas du mapper.

7.4 Mise à jour du contrat : Interface WikiRepository

Par analogie:

- Le service (WikiService) = le chef en cuisine.

- WikiRepository = la “fiche fournisseur” / le “catalogue d’approvisionnement” : le chef dit “je veux sauvegarder une page / récupérer une page”, sans savoir si ça vient du frigo A, du frigo B, ou du grossiste.

- DatabaseWikiRepository (plus tard) = le commis qui va réellement au frigo (DB) chercher/poser les ingrédients.

Ouvre le fichier et remplace le contenu par le code qui suit :

api/src/main/java/fr/mycellius/repository/WikiRepository.java

```
package fr.mycellius.repository;

import fr.mycellius.domain.WikiPage;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;

public interface WikiRepository {
    WikiPage save(WikiPage page);
    WikiPage getById(String id);
    Page<WikiPage> findAll(Pageable pageable);
    Page<WikiPage> searchByTitle(String title, Pageable pageable);
}
```

Lance : mvn -f api/pom.xml test

Tu devrais encore avoir des bugs... car on avait déjà une ancienne version de “repository layer” (avec des méthodes différentes), et on vient d’imposer un nouveau contrat (méthodes avec Pageable, etc.). Il faut donc aligner 3 autres fichiers qui appellent/implémentent ce contrat:

- WikiService.java
- InMemoryWiki.java
- DatabaseWikiRepository.java

Remplace ton fichier `api/src/main/java/fr/mycellius/service/WikiService.java` par :

```
package fr.mycellius.service;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.domain.exception.PageNotFoundException;
import fr.mycellius.repository.WikiRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

@Service
public class WikiService {

    private final WikiRepository repository;

    public WikiService(WikiRepository repository) {
        this.repository = repository;
    }

    public WikiPage createPage(WikiPage page) {
        if (page == null) {
            throw new IllegalArgumentException("La page est obligatoire");
        }

        // Règle métier : id unique
        WikiPage existing = repository.getById(page.getId());
        if (existing != null) {
            throw new IllegalArgumentException("Une page avec l'id " + page.getId() + " existe déjà");
        }

        return repository.save(page);
    }

    public WikiPage getPageById(String id) {
        WikiPage page = repository.getById(id);
        if (page == null) {
            throw new PageNotFoundException(id);
        }
        return page;
    }

    public Page<WikiPage> listPages(Pageable pageable) {
        return repository.findAll(pageable);
    }

    public Page<WikiPage> searchByTitle(String fragment, Pageable pageable) {
        return repository.searchByTitle(fragment, pageable);
    }
}
```

Lance : mvn -f api/pom.xml test

Dans api/src/main/java/fr/mycellius/repository/InMemoryWiki.java :

```
package fr.mycellius.repository;

import fr.mycellius.domain.WikiPage;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageImpl;
import org.springframework.data.domain.Pageable;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class InMemoryWiki implements WikiRepository {

    private final Map<String, WikiPage> store = new ConcurrentHashMap<>();

    @Override
    public WikiPage save(WikiPage page) {
        store.put(page.getId(), page);
        return page;
    }

    @Override
    public WikiPage getByid(String id) {
        return store.get(id);
    }

    @Override
    public Page<WikiPage> findAll(Pageable pageable) {
        List<WikiPage> all = new ArrayList<>(store.values());
        all.sort(Comparator.comparing(WikiPage::getId)); // simple et stable

        return toPage(all, pageable);
    }

    @Override
    public Page<WikiPage> searchByTitle(String title, Pageable pageable) {
        String needle = title == null ? "" : title.toLowerCase();
        List<WikiPage> filtered = store.values().stream()
            .filter(p -> p.getTitle() != null && p.getTitle().toLowerCase().contains(needle))
            .sorted(Comparator.comparing(WikiPage::getId))
            .toList();

        return toPage(filtered, pageable);
    }
}
```

```

    }

    private Page<WikiPage> toPage(List<WikiPage> source, Pageable pageable) {
        int start = (int) pageable.getOffset();
        int end = Math.min(start + pageable.getPageSize(), source.size());
        List<WikiPage> content = (start >= source.size()) ? List.of() : source.subList(start, end);
        return new PageImpl<>(content, pageable, source.size());
    }
}

```

Lance : mvn -f api/pom.xml test

Supprime : api/src/main/java/fr/mycellius/repository/DatabaseWikiRepository.java :

Lance : mvn -f api/pom.xml test

Oui je sais il y a pleins de bugs....

On a supprimé l'ancien DatabaseWikiRepository, donc maintenant c'est ton contrat de service qui est "le nouveau chef d'orchestre" ... et ton contrôleur est resté sur l'ancienne signature.

Cause exacte des 2 erreurs :

- WikiService.createPage(...) attend maintenant un seul objet WikiPage (pas 4 paramètres).
- WikiService.searchByTitle(...) attend maintenant (fragment, Pageable) (pas juste fragment).

A) Corriger la méthode POST

Dans ton WikiPageController, remplace le bloc :

```

List<Tag> domainTags = mapper.toDomainTags(request.tags());

WikiPage page = wikiService.createPage(
    request.id(),
    request.title(),
    request.content(),
    domainTags
);
return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toResponse(page));

```

par :

```

WikiPage created = wikiService.createPage(mapper.toDomain(request));
return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toResponse(created));

```

=> Important : avec MapStruct, mapper.toDomain(request) fabrique la "fiche interne" WikiPage à partir du "bon de commande" CreateWikiPageRequest.

B) Corriger la méthode GET /search

Tu as actuellement :

```
public List<WikiPageResponse> searchByTitle(@RequestParam("title") String fragment) {  
    return wikiService.searchByTitle(fragment).stream()  
        .map(mapper::toResponse)  
        .toList();  
}
```

Remplace par une version "paginée" (minimum pour compiler et alignée avec ton nouveau service) :

```
@GetMapping("/search")  
public org.springframework.data.domain.Page<WikiPageResponse> searchByTitle(  
    @RequestParam("title") String fragment,  
    @RequestParam(defaultValue = "0") int page,  
    @RequestParam(defaultValue = "10") int size  
) {  
    org.springframework.data.domain.PageRequest pageable =  
        org.springframework.data.domain.PageRequest.of(page, size);  
  
    return wikiService.searchByTitle(fragment, pageable).map(mapper::toResponse);  
}
```

C) Nettoyage imports (optionnel mais conseillé)

Après A + B, tu n'as normalement plus besoin de Tag ni List dans ce contrôleur.

Lance : mvn -f api/pom.xml test

On va maintenant corriger les tests :

On a changé le contrat de WikiService (et du repository) mais tes tests appellent encore l'ancienne signature. Du coup la compilation des tests casse. Donc il faut juste mettre les tests au nouveau contrat.

Fichier : api/src/test/java/fr/mycellius/service/WikiServiceTest.java

```
package fr.mycellius.service;  
  
import fr.mycellius.domain.Tag;  
import fr.mycellius.domain.WikiPage;  
import fr.mycellius.domain.exception.PageNotFoundException;  
import fr.mycellius.repository.InMemoryWiki;  
import org.junit.jupiter.api.Test;  
import org.springframework.data.domain.PageRequest;  
  
import java.util.List;
```

```

import static org.junit.jupiter.api.Assertions.*;

public class WikiServiceTest {

    // Petite méthode utilitaire pour fabriquer une Liste de tags de test
    private List<Tag> tagsDocker() {
        return List.of(new Tag("Docker"));
    }

    @Test
    void createPage_nominal_cree_une_page() {
        InMemoryWiki repo = new InMemoryWiki();
        WikiService service = new WikiService(repo);

        WikiPage page = service.createPage(new WikiPage("PAGE-001", "Installation Docker", "contenu",
tagsDocker()));

        assertEquals("PAGE-001", page.getId());
        assertEquals("Installation Docker", page.getTitle());

        // repo.findAll() attend maintenant un Pageable
        assertEquals(1, repo.findAll(PageRequest.of(0, 10)).getContent().size());
    }

    @Test
    void createPage_refuse_doublon_id() {
        InMemoryWiki repo = new InMemoryWiki();
        WikiService service = new WikiService(repo);

        service.createPage(new WikiPage("PAGE-001", "Titre 1", "contenu", tagsDocker()));

        assertThrows(IllegalArgumentException.class, () ->
            service.createPage(new WikiPage("PAGE-001", "Titre 2", "autre contenu", tagsDocker())))
        );
    }

    @Test
    void getPageById_page_existante_ok() {
        InMemoryWiki repo = new InMemoryWiki();
        WikiService service = new WikiService(repo);

        service.createPage(new WikiPage("PAGE-001", "Titre", "contenu", tagsDocker()));

        WikiPage page = service.getPageById("PAGE-001");

        assertEquals("PAGE-001", page.getId());
    }

    @Test

```

```

void getPageById_page_inexistante_declenche_exception() {
    InMemoryWiki repo = new InMemoryWiki();
    WikiService service = new WikiService(repo);

    assertThrows(PageNotFoundException.class, () ->
        service.getPageById("PAGE-999")
    );
}

@Test
void searchByTitle_retourne_liste_vide_si_rien() {
    InMemoryWiki repo = new InMemoryWiki();
    WikiService service = new WikiService(repo);

    // searchByTitle attend maintenant (fragment, pageable)
    List<WikiPage> result = service.searchByTitle("docker", PageRequest.of(0, 10)).getContent();

    assertTrue(result.isEmpty());
}

@Test
void searchByTitle_trouve_pages_correspondantes() {
    InMemoryWiki repo = new InMemoryWiki();
    WikiService service = new WikiService(repo);

    service.createPage(new WikiPage("PAGE-001", "Installation Docker", "...", tagsDocker()));
    service.createPage(new WikiPage("PAGE-002", "Docker avancé", "...", tagsDocker()));

    // searchByTitle attend maintenant (fragment, pageable)
    List<WikiPage> result = service.searchByTitle("docker", PageRequest.of(0, 10)).getContent();

    assertEquals(2, result.size());
}
}

```

Lance : `mvn -f api/pom.xml test`

Et là, normalement, tout devrait être OKKKK

7.5 DatabaseWikiRepository (implémentation DB + résolution tags)

Re-cr  er le fichier :

api/src/main/java/fr/mycellius/persistence/adapters/DatabaseWikiRepository.java

```
package fr.mycellius.persistence.adapters;

import fr.mycellius.domain.Tag;
import fr.mycellius.domain.WikiPage;
import fr.mycellius.persistence.entity.TagEntity;
import fr.mycellius.persistence.entity.WikiPageEntity;
import fr.mycellius.persistence.jpa.TagJpaRepository;
import fr.mycellius.persistence.jpa.WikiPageJpaRepository;
import fr.mycellius.persistence.mapper.WikiPageEntityMapper;
import fr.mycellius.repository.WikiRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Repository;

import java.time.Instant;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@Repository
public class DatabaseWikiRepository implements WikiRepository {

    private final WikiPageJpaRepository pageJpa;
    private final TagJpaRepository tagJpa;
    private final WikiPageEntityMapper mapper;

    public DatabaseWikiRepository(WikiPageJpaRepository pageJpa,
                                TagJpaRepository tagJpa,
                                WikiPageEntityMapper mapper) {
        this.pageJpa = pageJpa;
        this.tagJpa = tagJpa;
        this.mapper = mapper;
    }

    @Override
    public WikiPage save(WikiPage page) {
        WikiPageEntity entity = mapper.toEntity(page);

        // createdAt c  t   DB : si absent on le fixe maintenant
        if (entity.getCreatedAt() == null) {
            entity.setCreatedAt(Instant.now());
        }

        // tags : "find or create" (normalisation + d  doublonnage)
```

```

List<Tag> pageTags = (page.getTags() == null) ? List.of() : page.getTags();

Set<TagEntity> resolvedTags = pageTags.stream()
    .map(Tag::getValue)
    .map(v -> v == null ? "" : v.trim())
    .filter(v -> !v.isBlank())
    .map(String::toLowerCase)
    .distinct()
    .map(this::findOrCreateTag)
    .collect(Collectors.toSet());

entity.setTags(resolvedTags);

WikiPageEntity saved = pageJpa.save(entity);
return mapper.toDomain(saved);
}

private TagEntity findOrCreateTag(String value) {
    return tagJpa.findByValue(value)
        .orElseGet(() -> tagJpa.save(new TagEntity(value)));
}

@Override
public WikiPage getById(String id) {
    return pageJpa.findById(id).map(mapper::toDomain).orElse(null);
}

@Override
public Page<WikiPage> findAll(Pageable pageable) {
    return pageJpa.findAll(pageable).map(mapper::toDomain);
}

@Override
public Page<WikiPage> searchByTitle(String title, Pageable pageable) {
    return pageJpa.findByTitleContainingIgnoreCase(title, pageable).map(mapper::toDomain);
}
}

```

Lance : `mvn -f api/pom.xml test`

Explication “relation fonctionnelle” (séance 5 reprise, mais enrichie tags) :

- Le service parle à WikiRepository (contrat).
- DatabaseWikiRepository traduit le domaine en entity, appelle JPA, et retraduit en domaine.
- Pour les tags, DatabaseWikiRepository fait une étape en plus : “résoudre un tag unique” (find or create).

8. Service métier : il ne connaît ni DTO ni Entity

But : garder le service stable et testable, comme dans les séances 3–4.

Service métier (WikiService) doit parler à WikiRepository (contrat), pas à JPA directement.

Fichier : `api/src/main/java/fr/mycellius/service/WikiService.java` (on a déjà modifier ce fichier)

On vérifie juste que :

- Votre WikiService a bien un constructeur : `public WikiService(WikiRepository repository)`
- Et vos méthodes utilisent uniquement `repository.*` (`save/getById/findAll/searchByTitle`), pas de `InMemoryWiki` ni de `WikiPageJpaRepository` dedans.

=> Si c'est déjà le cas : ne changez rien.

Contrôleurs REST : DTO validé → service → DTO

9. On applique la logique de séance 4 : contrôleur = adaptateur HTTP.

9.1 Pages : POST/GET + pagination + search (normalement, on l'a déjà ce fichier mais il faut faire des modifications... je te laisse voir les différences. Il y a un peu de refactoring...)

Fichier : api/src/main/java/fr/mycellius/web/controller/WikiPageController.java

```
package fr.mycellius.web.controller;

import fr.mycellius.domain.WikiPage;
import fr.mycellius.service.WikiService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import fr.mycellius.web.dto.CreateWikiPageRequest;
import jakarta.validation.Valid;
import fr.mycellius.web.mapper.WikiPageDtoMapper;
import fr.mycellius.web.dto.WikiPageResponse;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;

@RestController
@RequestMapping("/api/v1/pages")
public class WikiPageController {

    private final WikiService wikiService;
    private final WikiPageDtoMapper mapper;

    public WikiPageController(WikiService wikiService, WikiPageDtoMapper mapper) {
        this.wikiService = wikiService;
        this.mapper = mapper;
    }

    @PostMapping
    public ResponseEntity<WikiPageResponse> createPage(@Valid @RequestBody CreateWikiPageRequest request) {

        WikiPage created = wikiService.createPage(mapper.toDomain(request));
        return ResponseEntity.status(HttpStatus.CREATED).body(mapper.toResponse(created));
    }

    @GetMapping("/{id}")
    public WikiPageResponse getPageById(@PathVariable String id) {
        return mapper.toResponse(wikiService.getPageById(id));
    }

    @GetMapping("/search")
    public Page<WikiPageResponse> searchByTitle(
        @RequestParam("title") String fragment,
        @RequestParam(defaultValue = "0") int page,
```

```

        @RequestParam(defaultValue = "10") int size
    ) {
        PageRequest pageable = PageRequest.of(page, size);

        return wikiService.searchByTitle(fragment, pageable).map(mapper::toResponse);
    }
}

```

Spring construit automatiquement les routes complètes :

```

@GetMapping("/{id}") devient GET /api/v1/pages/{id}
@GetMapping("/search") devient GET /api/v1/pages/search
@PostMapping devient POST /api/v1/pages

```

=====

Concrètement, WikiPageController.java sert de “porte d’entrée HTTP” de ton API pour la ressource WikiPage.

Il fait 3 choses, et uniquement ça (c’est sa responsabilité) :

1. Recevoir des requêtes HTTP (Postman / navigateur / front)

Exemples :

```

POST /api/v1/pages avec un JSON
GET /api/v1/pages/PAGE-001
GET /api/v1/pages/search?title=docker&page=0&size=10

```

2. Traduire le “langage HTTP” vers le “langage métier”

C’est là que tu utilises :

```

CreateWikiPageRequest = le formulaire JSON d’entrée
@Valid = le contrôle du formulaire (champs obligatoires, tailles, format...)
WikiPageDtoMapper = la conversion DTO → WikiPage (objet métier)

```

=> Le contrôleur ne fait pas le métier.

=> Il prépare juste les bons objets pour que le service travaille.

3. Renvoyer une réponse HTTP propre au client

Il renvoie :

un code HTTP (201, 200, 400, 404...)

un JSON au format “API” (WikiPageResponse), pas l’objet métier brut

4. Analogie restaurant :

> Le client parle au serveur (HTTP).

> Le serveur ne cuisine pas : il prend la commande, vérifie qu'elle est complète, la transmet en cuisine.

> La cuisine = WikiService (règles métier).

> Le serveur revient avec une assiette présentable (DTO de sortie) + "statut" (200/201).

=> Donc WikiPageController, c'est le serveur de salle : il gère l'échange entre l'extérieur (HTTP) et l'intérieur (métier), sans mélanger les rôles.

9.2 Tags : GET /api/v1/tags

Crée le fichier suivant: api/src/main/java/fr/mycellius/web/controller/TagController.java

```
package fr.mycellius.web.controller;

import fr.mycellius.persistence.entity.TagEntity;
import fr.mycellius.persistence.jpa.TagJpaRepository;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class TagController {

    private final TagJpaRepository tagJpa;

    public TagController(TagJpaRepository tagJpa) {
        this.tagJpa = tagJpa;
    }

    @GetMapping("/api/v1/tags")
    public List<String> listTags() {
        return tagJpa.findAll().stream()
            .map(TagEntity::getValue)
            .toList();
    }
}
```

Relance :

```
mvn -f api/pom.xml test
```

Puis démarre l'API :

```
mvn -f api/pom.xml spring-boot:run
```

Ensuite, test Postman :

GET <http://localhost:8080/api/v1/tags>

=> Attendu :

[] (si aucun tag n'a encore été créé en base),

ou

une liste de strings si tu as déjà fait des POST qui ont persisté des tags.

10. Erreurs standardisées + erreurs de validation DTO

En séance 4, tu as `ApiErrorResponse` + `ApiExceptionHandler` pour 400/404.

En séance 6, tu ajoutes : “quand un DTO est invalide, renvoyer un 400 structuré avec détails par champ”.

L'étape 10 ne sert pas à “faire marcher l'API” quand tout est nominal. *Elle sert à rendre l'API exploitable quand ça ne l'est pas (erreurs)*, et à stabiliser le contrat côté client.

En prod, ce n'est pas “tout fonctionne” qui coûte cher, c'est “ça casse et on ne comprend pas pourquoi”.

Concrètement, à quoi ça sert (très concret) :

a. Rendre les erreurs de validation lisibles pour le client

- Sans l'étape 10, si tu envoies un JSON invalide (title vide, mauvais format d'id, etc.), Spring renvoie souvent un 400 générique. Le front reçoit “Bad Request” sans savoir quel champ est fautif.

- Avec l'étape 10, tu reçois un 400 structuré, avec une liste “details” du type :

=> field = title, message = “Le titre est obligatoire”

=> Ça permet au front d'afficher une erreur sous le champ concerné, sans deviner.

b. Avoir un format d'erreur unique et stable sur toute l'API

- Tu standardises les erreurs : 404 (`PageNotFoundException`), 400 (`IllegalArgumentException`), 400 validation.

- Sans ça, selon l'erreur, le JSON peut changer (ou parfois pas de JSON), ce qui oblige à gérer 15 formats différents côté client.

Là, le client sait : “si erreur, je lis `ApiErrorResponse`”.

c. Déboguer plus vite

- Quand tu as un 400, le “path + timestamp + message + details” te dit immédiatement :

- sur quelle route ça a cassé

- quand

- pourquoi exactement

=> Ça évite de “rejouer” à l'aveugle dans Postman.

10.1 Faire évoluer ApiResponse

ApiResponse, c'est le "format standard" de réponse d'erreur de ton API.

Concrètement :

- Ça définit les champs qu'on renverra toujours quand il y a une erreur (error, message, status, path, timestamp).
- Ça permet d'avoir une réponse lisible et stable pour Postman / front (au lieu d'un 400 "brut" qui change selon Spring).
- Et ça prévoit un cas spécial pour la validation (details) : une liste "champ → message" pour dire précisément pourquoi le JSON est invalide.

Les deux méthodes statiques servent juste à fabriquer facilement l'objet :

- **of(...)** : erreur "classique" (404, 400 métier...) avec details=[].
- **validation(...)** : erreur de validation avec error="VALIDATION_ERROR" + details rempli.

Crée ou met à jour le fichier : api/src/main/java/fr/mycellius/web/ApiResponse.java

```
package fr.mycellius.web;

import java.time.Instant;
import java.util.List;

public record ApiResponse(
    String error,
    String message,
    int status,
    String path,
    String timestamp,
    List<FieldError> details
) {
    public record FieldError(String field, String message) {}

    public static ApiResponse of(String error, String message, int status, String path) {
        return new ApiResponse(
            error,
            message,
            status,
            path,
            Instant.now().toString(),
            List.of()
        );
    }

    public static ApiResponse validation(String message, int status, String path, List<FieldError> details) {
        return new ApiResponse(
```

```

        "VALIDATION_ERROR",
        message,
        status,
        path,
        Instant.now().toString(),
        details == null ? List.of() : details
    );
}
}

```

10.2 Ajouter la gestion des erreurs de validation `MethodArgumentNotValidException` dans `ApiExceptionHandler`

Objectif :

- quand ton DTO est invalide (`@Valid`), Spring déclenche *MethodArgumentNotValidException*.
- Sans handler, tu as un 400 "pas très exploitable".
- Avec le handler, tu renvoies un JSON propre au format `ApiErrorResponse` avec *details*.

Ouvre le fichier à modifier et remplace le avec le code suivant (la base était bonne mais on y rajoute des handlers) :

`api/src/main/java/fr/mycellius/web/ApiExceptionHandler.java`

```

package fr.mycellius.web;

import fr.mycellius.domain.exception.PageNotFoundException;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.List;

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(PageNotFoundException.class)
    public ResponseEntity<ApiErrorResponse> handlePageNotFound(PageNotFoundException ex,
        HttpServletRequest request) {

        ApiErrorResponse body = ApiErrorResponse.of(
            "PAGE_NOT_FOUND",
            ex.getMessage(),
            HttpStatus.NOT_FOUND.value(),
            request.getRequestURI()
        );
    }
}

```



```

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ApiErrorResponse> handleIllegalArgumentException(IllegalArgumentException ex,
    HttpServletRequest request) {

        ApiErrorResponse body = ApiErrorResponse.of(
            "INVALID_INPUT",
            ex.getMessage(),
            HttpStatus.BAD_REQUEST.value(),
            request.getRequestURI()
        );
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }

    //  NOUVEAU : erreurs de validation (@Valid)
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiErrorResponse> handleValidation(MethodArgumentNotValidException ex,
    HttpServletRequest request) {

        List<ApiErrorResponse.FieldError> details = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(f -> new ApiErrorResponse.FieldError(f.getField(), f.getDefaultMessage()))
            .toList();

        ApiErrorResponse body = ApiErrorResponse.validation(
            "Le JSON envoyé est invalide",
            HttpStatus.BAD_REQUEST.value(),
            request.getRequestURI(),
            details
        );

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }
}

```

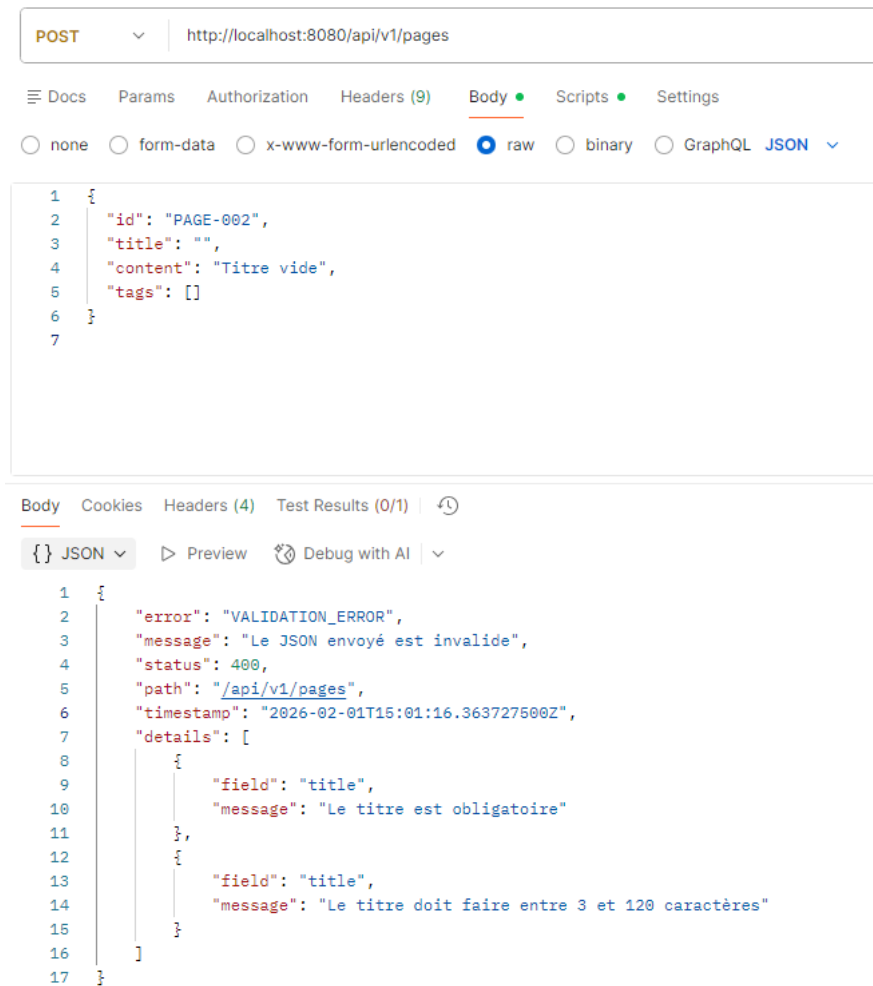
Lance : `mvn -f api/pom.xml test` => Résultat attendu : **BUILD SUCCESS**.

Test Postman validation (le test “preuve”): Fais un POST sur /api/v1/pages avec un JSON invalide, par ex. :

```
{
  "id": "PAGE-002",
  "title": "",
  "content": "Titre vide",
  "tags": []
}
```

Attendu :

- HTTP 400
- body JSON avec :
 - error: "VALIDATION_ERROR"
 - contenant au moins title avec un message du genre “Le titre est obligatoire” (selon tes annotations)



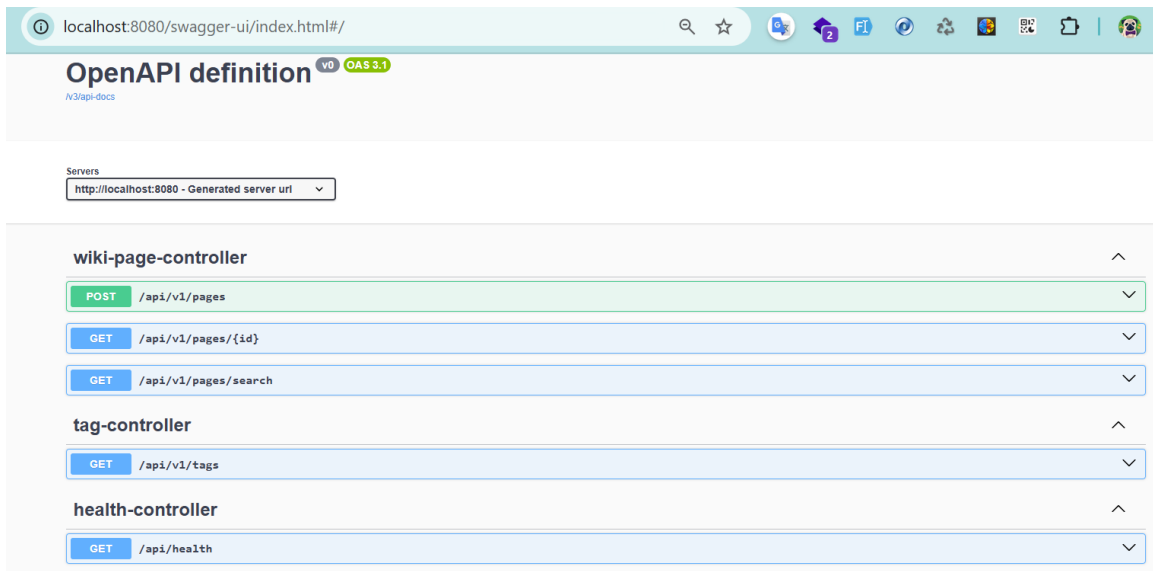
11. Swagger/OpenAPI : preuve rapide

Avec Springdoc ajouté, tu dois pouvoir ouvrir Swagger UI (selon config springdoc) :

`/swagger-ui.html` ou `/swagger-ui/index.html`

Tu t'en sers comme “preuve” que le contrat d’API est compréhensible, et que les DTO/structures existent.

Attendu : tu vois l’UI Swagger avec tes endpoints (au minimum `/api/v1/pages` et `/api/v1/tags`).



12. Tests manuels Postman (rituel séance 4, mais sur v1)

Même esprit que la séance 4 : collection Postman, baseUrl, puis requêtes.

12.1 POST OK (création avec tags)

Requête :

- Method: POST
- URL: <http://localhost:8080/api/v1/pages>
- Headers: Content-Type: application/json
- Body (raw / JSON) :

```
POST {{baseUrl}}/api/v1/pages

{
  "id": "PAGE-001",
  "title": "Bienvenue dans Mycellius",
  "content": "Première page persistée avec tags.",
  "tags": [
    { "name": "onboarding" },
    { "name": "dev" }
  ]
}
```

Attendu :

- Status 201 - Created
- Body = WikiPageResponse (tu dois au minimum voir id/title/content, et selon ton mapper actuel tags/createdAt peuvent encore être vides/null si pas encore câblés côté DTO)

12.2 POST invalide (validation)

```
{
  "id": "PAGE-002",
  "title": "",
  "content": "Titre vide",
  "tags": []
}
```

Attendu : 400 + ApiErrorResponse + details sur title (comme l'idée "400 données invalides" vue en séance 4, mais enrichie validation).

12.3 GET by id

GET {{baseUrl}}/api/v1/pages/PAGE-001 → 200 + Un JSON WikiPageResponse (tu verras id/title/content + tags selon ton mapping actuel, et createdAt selon ton DTO/mapper)

12.4 Pagination

GET {{baseUrl}}/api/v1/pages?page=0&size=2 → 405 + error

Pourquoi ? C'est juste parce qu'on n'a pas de @GetMapping "racine" sur /api/v1/pages et aucune méthode GET n'est déclarée dessus donc ⇒ 405 Method Not Allowed.

Actuellement tu as :

- POST /api/v1/pages
- GET /api/v1/pages/{id}
- GET /api/v1/pages/search

On va faire le nécessaire :

Dans WikiPageController.java, ajoute ce code (en gardant tes imports Page et PageRequest) :

```
@GetMapping
public Page<WikiPageResponse> list(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size
) {
    PageRequest pageable = PageRequest.of(page, size);
    return wikiService.listPages(pageable).map(mapper::toResponse);
}
```

=> Relance l'API et re-test la requête GET -> 200 + un JSON de type Page

```
{
  "content": [
    {
      "id": "PAGE-001",
      "title": "Bienvenue dans Mycellius",
      "content": "Première page persistée avec tags.",
      "tags": [],
      "createdAt": null
    },
    {
      "id": "PAGE-101",
      "title": "Test tags",
      "content": "xxx",
      "tags": [],
      "createdAt": null
    }
  ],
  "pageable": {
```

```

    "pageNumber": 0,
    "pageSize": 2,
    "sort": {
      "empty": true,
      "sorted": false,
      "unsorted": true
    },
    "offset": 0,
    "paged": true,
    "unpaged": false
  },
  "last": false,
  "totalElements": 3,
  "totalPages": 2,
  "size": 2,
  "number": 0,
  "sort": {
    "empty": true,
    "sorted": false,
    "unsorted": true
  },
  "first": true,
  "numberOfElements": 2,
  "empty": false
}

```

12.5 Recherche titre

GET {{baseUrl}}/api/v1/pages/search?title=bien&page=0&size=10 → 200 + Page JSON.

(Aligné avec l'idée /search?title=... de la séance 4.)

12.6 Liste tags

GET {{baseUrl}}/api/v1/tags → 200 + ["onboarding","dev"...]

```

[
  "dev",
  "java",
  "onboarding"
]

```

13. Relations fonctionnelles entre fichiers

Flux complet (du JSON jusqu'à MySQL, puis retour) :

1. HTTP → Controller

WikiPageController reçoit le JSON dans CreateWikiPageRequest

@Valid déclenche Jakarta Validation

2. DTO → Domaine (MapStruct web)

WikiPageDtoMapper.toDomain() transforme CreateWikiPageRequest en WikiPage (métier)

3. Domaine → Service

WikiService.createPage(WikiPage) applique les règles métier (et reste testable, indépendamment de Spring/DB)

4. Service → Repository (contrat)

WikiRepository est le contrat de stockage (comme en séance 5).

5. Repository DB → Entity/JPA/MySQL

- DatabaseWikiRepository traduit WikiPage → WikiPageEntity (MapStruct persistance)
- résout tags via TagJpaRepository (find or create)
- sauvegarde via WikiPageJpaRepository vers MySQL
- retraduit WikiPageEntity → WikiPage

6. Domaine → DTO (MapStruct web)

WikiPageDtoMapper.toResponse() renvoie un WikiPageResponse front-friendly

7. Exceptions → HTTP (handler global)

ApiExceptionHandler traduit :

- PageNotFoundException → 404
- IllegalArgumentException → 400
- MethodArgumentNotValidException → 400 + details (validation DTO)

=> C'est le prolongement direct de la séance 4.

14. Fin de séance : Git + CI

Rappel rituel : compiler/tester, add/commit/push, vérifier CI, PR dev → test.

```
mvn -f api/pom.xml test
```

```
git status
```

```
git add api/
```

```
git commit -m "feat(api): add dto validation + mapstruct + tags + pagination"
```

```
git push origin dev
```

=> Ensuite sur GitHub : vérifier Actions en vert, PR dev → test.